

Towards Energy-Efficient Brain-Inspired AI: An Overview and Evaluation of Spiking Neural Networks

Julen Legido Núñez



Universitat
Pompeu Fabra
Barcelona

Towards Energy-Efficient Brain-Inspired AI: An Overview and Evaluation of Spiking Neural Networks

TREBALL DE FI DE GRAU DE
Julen Legido Núñez

Francesc Wilhelmi

Grau en Enginyeria en Informàtica

Curs 2025 - 2026



Universitat
Pompeu Fabra
Barcelona

Escola
d'Enginyeria

Acknowledgements

I would like to express my sincere gratitude to my thesis supervisor, Francesc Wilhelmi, for proposing the research topic that inspired this work and introducing me to it. His guidance, support and valuable feedback throughout every stage of the thesis were essential to its development and completion.

I would also like to thank my family, friends and colleagues for their constant encouragement and unconditional support throughout my academic journey. Their confidence in me has always been a source of motivation.

Abstract

Spiking Neural Networks (SNNs) have emerged as a promising alternative to conventional Artificial Neural Networks (ANNs) due to their biologically inspired computation and potential for energy-efficient processing. Unlike traditional neural networks, SNNs communicate through discrete spike events distributed over time, enabling sparse and event-driven information processing.

This thesis investigates the behavior of SNNs in comparison with ANN models across both static and event-based image classification tasks. Experiments were conducted using MNIST, CIFAR-10, N-MNIST, and CIFAR10-DVS datasets, evaluating not only classification accuracy but also spike activity metrics, temporal dynamics, and computational efficiency. Several encoding strategies, architectural modifications, and optimization techniques were explored, including rate encoding, gradient-based encoding, normalization methods, and temporal parameter tuning.

The results show that SNNs can achieve competitive performance, particularly on datasets containing meaningful temporal information. However, challenges related to input encoding, optimization stability, and hardware execution remain significant. A theoretical operation-count analysis further suggests that the computational advantages of SNNs become most evident when sparse spike activity is executed on dedicated neuromorphic hardware. Overall, the work highlights both the current limitations and future potential of spike-based neural computation.

Resum

Les Xarxes Neuronals de Polsos (Spiking Neural Networks, SNNs) han emergit com una alternativa prometedora a les Xarxes Neuronals Artificials (ANNs) convencionals gràcies a la seva inspiració biològica i al seu potencial per oferir un processament més eficient energèticament. A diferència de les xarxes neuronals tradicionals, les SNNs es comuniquen mitjançant esdeveniments discrets de polsos distribuïts en el temps, fet que permet un processament de la informació dispers i basat en esdeveniments.

Aquest treball estudia el comportament de les SNNs en comparació amb models ANN en tasques de classificació d'imatges tant estàtiques com temporals. Els experiments es van realitzar utilitzant els conjunts de dades MNIST, CIFAR-10, N-MNIST i CIFAR10-DVS, avaluant no només la precisió de classificació, sinó també mètriques relacionades amb l'activitat neuronal, la dinàmica temporal i l'eficiència computacional. Igualment, es van analitzar diferents estratègies de codificació, modificacions arquitectòniques i tècniques d'optimització.

Els resultats mostren que les SNNs poden assolir un rendiment competitiu, especialment quan les dades contenen informació temporal significativa. Tot i això, encara existeixen reptes importants relacionats amb la codificació d'entrada, l'estabilitat de l'entrenament i l'execució sobre maquinari convencional. Finalment, l'anàlisi teòrica d'operacions suggereix que els principals avantatges computacionals de les SNNs es manifesten en maquinari neuromòrfic especialitzat.

Resumen

Las Redes Neuronales de Pulsos (Spiking Neural Networks, SNNs) han surgido como una alternativa prometedora a las Redes Neuronales Artificiales (ANNs) convencionales gracias a su inspiración biológica y a su potencial para realizar un procesamiento más eficiente energéticamente. A diferencia de las redes neuronales tradicionales, las SNNs se comunican mediante eventos discretos de pulsos distribuidos en el tiempo, permitiendo un procesamiento de la información disperso y dirigido por eventos.

En este trabajo se estudia el comportamiento de las SNNs en comparación con modelos ANN en tareas de clasificación de imágenes tanto estáticas como temporales. Los experimentos se realizaron utilizando los conjuntos de datos MNIST, CIFAR-10, N-MNIST y CIFAR10-DVS, evaluando no solo

la precisión de clasificación, sino también métricas relacionadas con la actividad neuronal, la dinámica temporal y la eficiencia computacional. Asimismo, se analizaron diferentes estrategias de codificación, modificaciones arquitectónicas y técnicas de optimización.

Los resultados muestran que las SNNs pueden alcanzar un rendimiento competitivo, especialmente cuando los datos contienen información temporal relevante. Sin embargo, persisten desafíos importantes relacionados con la codificación de entrada, la estabilidad del entrenamiento y la ejecución sobre hardware convencional. Finalmente, el análisis teórico de operaciones sugiere que las principales ventajas computacionales de las SNNs se manifiestan en hardware neuromórfico especializado.

Contents

List of Figures	ix
List of Tables	x
1 INTRODUCTION	1
1.1 Motivation	1
1.2 Research Questions	2
1.3 Contributions	2
2 STATE OF THE ART	3
2.1 Deep Learning and Modern Artificial Neural Networks	3
2.2 Spiking Neural Networks	3
2.3 Neuromorphic Computing	5
3 METHODOLOGY	7
3.1 Experimental Framework	7
3.2 Datasets	7
3.3 Considered Models	8
3.3.1 ANN Models	8
3.3.2 SNN Models	8
3.4 Information Encoding and Decoding	9
3.4.1 Rate Encoding	9
3.4.2 Latency Encoding	9
3.4.3 Gradient Encoding	10
3.4.4 Center-Weighted Encoding	10
3.4.5 Spike Probability Scaling	10
3.4.6 Output Decoding	10
3.5 Training Methodology	11
3.5.1 Loss Functions	11
3.5.2 Reproducibility and Seed Experiments	11
3.5.3 Data Splitting and Evaluation Protocol	11
3.6 Evaluation Metrics	11
3.6.1 Performance Metrics	11
3.6.2 Computational Metrics	11
4 PERFORMANCE EVALUATION	13
4.1 Static Dataset Experiments	13
4.1.1 MNIST	13
4.1.2 CIFAR-10	17
4.1.3 Main Findings on Static Dataset Experimentations	21
4.2 Temporal Dataset Experiments	22
4.2.1 N-MNIST	22

4.2.2	CIFAR10-DVS	22
4.3	Spike Activity Analysis	24
4.3.1	Sparsity and Firing Rate Trends	24
4.3.2	Relationship Between Spike Activity and Accuracy	24
4.3.3	Static vs. Temporal Datasets	25
4.4	Energy Analysis and Measurements	25
4.4.1	CodeCarbon Results and Interpretation	25
4.4.2	Operation-Count Analysis	26
4.5	Summary of Experiments	27
5	DISCUSSION	29
5.1	Threats to Validity	29
5.2	Performance Differences Between Static and Temporal Datasets	29
5.3	Relationship Between Sparsity and Learning	30
5.4	Optimization Challenges in SNNs	30
5.5	Accuracy-Efficiency Trade-Off	30
5.6	Hardware Dependency of SNN Efficiency	31
6	CONCLUSIONS	32
6.1	Main Findings	32
6.2	Future Work	33
A	DECLARATION OF ARTIFICIAL INTELLIGENCE USE	36

List of Figures

1.1	ANN (left) and SNN (right) structure comparison. <i>Source: IC-SNN: Optimal ANN2SNN Conversion at Low Latency, December 2022.</i>	2
2.1	Neuron models. <i>Source: https://snntorch.readthedocs.io/en/latest/index.html</i>	4
3.1	SNN encoding methods. <i>Source: https://snntorch.readthedocs.io/en/latest/index.html</i>	9
4.1	ANN and SNN accuracy and time per epoch comparison.	13
4.2	Spike analysis from the first SNN model.	14
4.3	Effect of different spike probabilities on accuracy and firing rate.	15
4.4	Center-Weighted encoding accuracy and spike activity results.	16
4.5	Accuracy and firing rate comparison between all encoding strategies	17
4.6	CIFAR-10 accuracy and time comparison.	18
4.7	Accuracy and sparsity on MSE and CE losses comparison.	18
4.8	Accuracy and time comparison between the model with and without normalization.	19
4.9	Kaiming accuracy and time comparison at timestep 15.	20
4.10	Deeper CNN accuracy and time comparison at timestep 15.	20
4.11	Accuracy and sparsity comparison between different β values.	21
4.12	ANN and SNN accuracies (a), SNN accuracy with different timesteps (b).	22
4.13	Accuracy and loss comparisons on CIFAR10-DVS.	23
4.14	Accuracy and firing rate with different β values.	24
4.15	Sparsity and firing rate values across all datasets.	24
4.16	Accuracy results across every dataset.	25

List of Tables

4.1	Performance of ANN (in green) vs. SNN (in blue) in CIFAR10-DVS, N-MNIST, and CIFAR-10. Energy consumption and carbon emissions correspond to the mean values obtained across three independent runs using different random seeds.	26
4.2	SNN metrics including timesteps, firing rate, estimated active Synaptic Operations (SOPs), and energy reduction.	27
4.3	Summary of the experimental findings and their implications.	27

Chapter 1

INTRODUCTION

1.1 Motivation

Since the emergence of Artificial Intelligence (AI), one of its most fascinating ambitions has been the creation of systems capable of learning and processing information with the human brain's efficiency. This is starting to materialize into reality through the rise of Artificial Neural Networks (ANNs) (McCulloch and Pitts, 1943; Rosenblatt et al., 1962). Such advances in multiple scientific and industrial fields have transformed the perception of what machines are capable of achieving, positioning deep learning as one of the leading technological breakthroughs of modern computing (Goodfellow et al., 2016).

Despite being originally inspired by the human brain, conventional ANNs remain fundamentally different from biological neural systems. Traditional deep learning models rely on dense numerical operations and continuous activations which require specific computational resources and significant energy consumption (Dayan and Abbott, 2001). On top of that, their continuous growth in size and complexity has motivated researchers to explore alternative architectures that may offer more efficient forms of computation while maintaining strong learning capabilities. Ultimately, the brain is still the ideal place to look for inspiration to develop more efficient neural networks.

In this context, Spiking Neural Networks (SNNs) represent one of the most promising directions in current AI research. Unlike traditional neural networks, which rely on synchronous, continuous-valued activations across all nodes, SNNs communicate through discrete spike events distributed over time, introducing temporal dynamics into the computational process (see figure 1.1). Instead of processing information continuously, neurons remain mostly idle and only emit spikes when activity occurs. This event-driven behavior resembles biological neural communication more closely, enabling sparse forms of computation that differ from standard ANN-based approaches (Maass, 1997; Roy et al., 2019). The ambition of SNNs goes further than simply being another neural architecture. They introduce different ways of understanding information representation, temporal encoding, asynchronous communication, and neural computation itself, considering other ways in which learning systems should process information, how time can be integrated into neural representations, or how computational efficiency can be achieved through sparsity rather than brute-force numerical processing.

The objective of this thesis is to study and experimentally analyze the behavior of SNNs in comparison with standard ANNs. These experiments are across different types of visual data, with particular emphasis not only on classification performance, but mainly on the internal characteristics of spike-based computation, including firing activity, sparsity, temporal processing, and computational implications. Through the implementation and evaluation of multiple architectures, this work aims to provide a deeper understanding of the practical behavior of SNNs, their current limitations, and their future potential within AI systems (Tavanaei et al., 2019; Eshraghian et al., 2023).

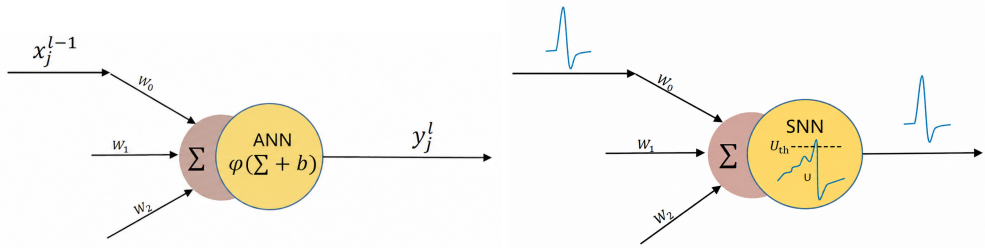


Figure 1.1: ANN (left) and SNN (right) structure comparison. *Source: IC-SNN: Optimal ANN2SNN Conversion at Low Latency, December 2022.*

1.2 Research Questions

This thesis seeks to investigate the following research questions:

1. How do ANNs and SNNs compare when evaluated on static and event-based visual classification datasets under comparable architectural conditions?
2. How do key SNN hyperparameters, including the number of timesteps, spike encoding strategies, membrane decay factors, and normalization techniques, influence classification performance and spike activity?
3. What empirical relationship exists between firing rate, sparsity, and predictive performance in SNNs?
4. To what extent do energy measurements obtained on conventional hardware align with, or differ from, theoretical efficiency estimates derived from neuromorphic computing principles?

Answering these questions will provide a comprehensive evaluation of the practical capabilities and limitations of contemporary SNNs, while also examining the extent to which their theoretical advantages can be observed under current software and hardware conditions.

1.3 Contributions

The contributions of this thesis are as follows:

- Implementation of ANN and SNN models for image recognition tasks.
- Evaluation of the provided models on MNIST, CIFAR-10, N-MNIST, and CIFAR10-DVS datasets.
- Investigation of the impact of key SNN design choices including spike encoding strategies, spike probability scaling, membrane leak factors and normalization techniques.
- Analysis of the internal dynamics of SNNs through spike-based metrics such as firing rate, sparsity and total spike activity.
- Evaluation of the practical computational cost of ANN and SNN models on conventional hardware using execution time, energy consumption and carbon-emission estimates obtained through CodeCarbon.
- A theoretical efficiency analysis providing an estimate of the potential computational advantages of SNNs under neuromorphic hardware.

Chapter 2

STATE OF THE ART

2.1 Deep Learning and Modern Artificial Neural Networks

ANNs constitute one of the most powerful tools in modern AI. Their success is primarily due to the development of deep learning architectures capable of learning complex patterns from vast datasets in a range of domains, including computer vision, speech recognition, natural language processing, and autonomous systems (Schmidhuber, 2015).

Among the different ANN architectures, Convolutional Neural Networks (CNNs) have become popular in computer vision tasks. CNNs exploit spatial locality through convolutional filters that automatically extract complex visual features from input data. Later on, the introduction of residual learning through ResNet architectures represented a big breakthrough in deep learning as it mitigated degradation problems associated with very deep networks (He et al., 2016). More recently, Transformer-based architectures have further transformed the field by replacing convolutional operations with attention mechanisms capable of modeling long-range dependencies efficiently (Dosovitskiy et al., 2021).

Training deep neural networks generally relies on backpropagation combined with gradient-based optimization algorithms such as Stochastic Gradient Descent (SGD), Adam, or RMSProp (Goodfellow et al., 2016). During training, gradients of the loss function are propagated backward through the network in order to iteratively update model parameters and minimize prediction error. These training operations are performed regardless of whether intermediate activations contain relevant information or remain close to zero, which entails a huge amount of Multiply-Accumulate (MAC) operations that involve millions of trainable parameters. Evidence indicates that performance improves when increasing parameter count, training data, and computational resources simultaneously (Kaplan et al., 2020). However, recent studies have highlighted the growing environmental and computational cost associated with large-scale deep learning models (Strubell et al., 2019). Training state-of-the-art architectures often requires extensive GPU clusters and high electrical consumption. Consequently, improving neural networks' computational efficiency has become an increasingly important research objective within modern AI systems.

Another important limitation concerns biological plausibility, as conventional neural networks process information synchronously, whereas biological neural systems communicate through sparse asynchronous electrical impulses distributed over time (Maass, 1997). That is, standard ANNs update every neuron in a layer simultaneously at fixed intervals, so the entire system constantly calculates and consumes power. In contrast, biological neurons act independently; they remain completely silent and only fire a spike when they receive enough input. This discrepancy has pushed towards the exploration of alternative computational paradigms inspired more directly by biological neural processing.

2.2 Spiking Neural Networks

SNNs represent a biologically-inspired neural computation paradigm in which information is transmitted through discrete electrical impulses known as spikes (Gerstner and Kistler, 2002). The neurons in SNNs

remain mostly inactive and only emit spikes when their internal membrane potential exceeds a predefined threshold (Maass, 1997). This spike-based communication mechanism is inspired by biological neural systems, where neurons exchange information through action potentials occurring asynchronously over time (Gerstner and Kistler, 2002). Thus, information in SNNs is represented not only through signal magnitude, but also through spike timing and firing frequency (Dayan and Abbott, 2001).

One of the most widely used neuron formulations within SNN research is the Leaky Integrate-and-Fire (LIF) neuron model (Gerstner and Kistler, 2002; Roy et al., 2019), represented in Figure 2.1. In this model, neurons accumulate incoming spikes in a membrane potential that gradually decays according to a leakage parameter. When the membrane potential exceeds a threshold value, the neuron emits a spike and resets its internal state (Gerstner et al., 2014). The membrane dynamics of a discrete LIF neuron can be expressed as

$$u_t = \beta u_{t-1} + \sum_j w_j x_j, \quad (2.1)$$

where u_t denotes the membrane potential at timestep t , β represents the membrane leak coefficient, w_j are the synaptic weights, and x_j corresponds to incoming spikes. Unlike conventional ANNs, where temporal dependencies often require recurrent architectures or sequence-processing mechanisms, SNNs naturally integrate temporal evolution through membrane dynamics and spike propagation (Tavanaei et al., 2019). The internal membrane potential evolves continuously across timesteps. Incoming spikes contribute incrementally to this potential, while the leak coefficient gradually decreases previously accumulated information over time. Consequently, each neuron possesses a form of short-term temporal memory, since its current state depends not only on the present input but also on previous spike activity. Spike propagation between neurons, therefore, allows information to be distributed sequentially through time rather than instantaneously as in conventional feedforward neural networks.

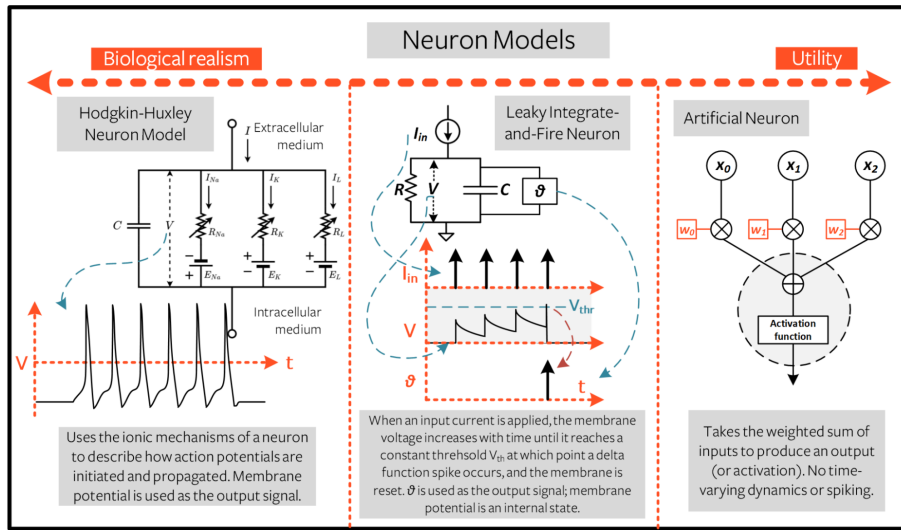


Figure 2.1: Neuron models. *Source:* <https://snntorch.readthedocs.io/en/latest/index.html>

One of the principal theoretical advantages of SNNs is computational sparsity. Since neurons only emit spikes occasionally, a large proportion of synaptic operations remain inactive during inference (Roy et al., 2019). This event-driven behavior potentially reduces unnecessary computation and enables sparse activity patterns both spatially and temporally.

This theoretical advantage has been demonstrated across several neuromorphic computing environments. A prominent use case is event-based computer vision, where SNNs process data directly from Neuromorphic Vision Sensors (NVS). Instead of analyzing static frames, these systems achieve extreme efficiency by computing features only when pixel-level intensity changes occur, demonstrating significant utility in low-latency robotic perception and autonomous tracking (Gallego et al., 2020; Gehrig

et al., 2020). Furthermore, the practical benefits of this temporal sparsity have been validated in human-machine interaction, such as fully event-based gesture recognition systems that operate at a fraction of the power required by conventional architectures (Amira et al., 2017). The growing interest in SNNs is also closely connected to the emergence of event-based sensing technologies. Traditional image sensors acquire visual information synchronously at fixed frame rates, producing dense image sequences regardless of scene dynamics. In contrast, event-based sensors operate asynchronously and only generate events when changes in brightness occur at individual pixels (Gallego et al., 2020). Dynamic Vision Sensors (DVS) constitute one of the most representative event-based sensing technologies. Instead of transmitting complete image frames, DVS cameras produce sparse streams of events characterized by spatial coordinates, timestamps, and polarity (Lichtsteiner et al., 2008), which naturally aligns with the temporal and event-driven nature of SNNs. These sensors provide several advantages compared to conventional cameras, including extremely high temporal resolution, low latency, reduced spatial redundancy, and improved dynamic range (Delbruck et al., 2010).

However, training deep SNNs remains considerably more difficult than conventional ANNs. The principal challenge arises from the non-differentiable nature of spike generation. Since spikes are binary events, standard backpropagation cannot directly compute derivatives of the spike activation function (Neftci et al., 2019). To address this limitation, modern research frequently relies on surrogate gradient methods. These approaches approximate the derivative of the spike function during backpropagation while preserving discrete spike generation during forward computation (Zenke and Vogels, 2021). Surrogate-gradient-based training has significantly improved the trainability of deep SNN architectures and enabled competitive performance on several benchmark datasets.

In parallel, ANN-to-SNN conversion techniques have also emerged as an alternative training strategy. In these methods, a pre-trained ANN is transformed into an equivalent spiking architecture while attempting to preserve predictive behavior (Rueckauer et al., 2017). Although conversion approaches can achieve strong inference performance, they often require long execution times and may introduce latency-related trade-offs.

The development of modern software frameworks has also accelerated accessibility to the field. Libraries such as `snnTorch`, `Norse`, `SpikingJelly`, and `BindSNET` simplify the implementation of surrogate gradient training, spike encoding, and temporal neural dynamics using modern deep learning ecosystems (Eshraghian et al., 2023). In addition, the growing availability of neuromorphic datasets has enabled systematic evaluation of SNN architectures on temporal visual tasks. Common benchmarks include N-MNIST¹ (Orchard et al., 2015), CIFAR10-DVS², DVS Gesture and SHD. However, event-based data processing introduces additional complexity. Event streams are frequently sparse and noisy, while temporal information must be encoded efficiently into spike-compatible representations (Amira et al., 2017).

Different temporal framing strategies, encoding mechanisms, and preprocessing techniques, which we will see later on, can substantially influence final model performance. Interestingly, several recent studies have shown that SNN architectures can achieve highly competitive performance compared to conventional CNNs applied to temporally aggregated event frames (Gehrig et al., 2020).

2.3 Neuromorphic Computing

Neuromorphic computing represents a hardware-oriented research field inspired by biological neural systems. It emerged as a research field during the late 1980s through the work of Carver Mead (Mead, 1990), who proposed designing electronic systems inspired directly by the organizational principles of biological neural systems. The objective of neuromorphic engineering was not only to imitate the structure of the brain, but also to reproduce its computational efficiency, parallelism, and low energy consumption.

¹<https://www.garrickorchard.com/datasets/n-mnist>

²<https://tonic.readthedocs.io/en/main/generated/tonic.datasets.CIFAR10DVS.html>

Neuromorphic processors attempt to exploit sparse spike-based communication directly at the hardware level using asynchronous and event-driven computation mechanisms (Mead, 1990). Neuromorphic hardware activates computation only when spike events occur, which potentially enables substantial reductions in energy consumption and latency for sparse inference workloads (Furber, 2016). Several neuromorphic hardware platforms have emerged during recent years: IBM TrueNorth³, Intel Loihi⁴, SpiNNaker⁵, and BrainScaleS⁶ are among the most representative systems (Merolla et al., 2014). Intel Loihi demonstrated particularly promising results regarding low-power inference and online learning capabilities (Davies et al., 2018). Similarly, SpiNNaker focused on large-scale simulation of biological neural systems through massively parallel asynchronous architectures.

As mentioned before, one of the main motivations behind neuromorphic computing is computational efficiency. Despite containing approximately 10^{11} neurons and 10^{15} synaptic connections, the human brain operates at an estimated power consumption of only around 20 W (Laughlin and Sejnowski, 2003). Conventional ANN hardware relies heavily on dense MACs, whereas spike-based systems primarily depend on simpler Synaptic Operations (SOPs) that are only activated when spikes occur. Theoretical analyses suggest that sparse spike activity could substantially reduce arithmetic energy consumption under appropriate hardware conditions (Roy et al., 2019). Nevertheless, neuromorphic computing remains an evolving field with several unresolved limitations. Software ecosystems remain considerably less mature than conventional deep learning frameworks, deployment pipelines are frequently hardware-specific, and portability between architectures remains limited (Schuman et al., 2022). An important distinction must therefore be made between the theoretical efficiency potential of SNNs and their practical execution on traditional von Neumann hardware. As we will see later, sparse spike activity of SNN experiments executed on GPUs cannot be fully exploited due to dense tensor computation and timestep simulation overhead (Davies et al., 2021).

³<https://research.ibm.com/publications/truenorth-design-and-tool-flow-of-a-65-mw-1-million-neuron-programmable-neurosynaptic-chip?>

⁴<https://www.intel.la/content/www/xl/es/research/neuromorphic-computing-loihi-2-technology-brief.html>

⁵<https://en.wikipedia.org/wiki/SpiNNaker>

⁶<https://ebrains.eu/data-tools-services/tools/brainscales>

Chapter 3

METHODOLOGY

In order to evaluate the behavior of ANNs and SNNs across static and event-based visual classification tasks, several experiments not only to compare predictive performance, but also to analyze the internal activity of SNNs were carried out. ANN and SNN architectures were implemented using comparable convolutional structures and evaluated using accuracy, loss, spike activity metrics, training time, and measured energy consumption. Image classification was selected because it constitutes one of the most established benchmark tasks in deep learning and allows direct comparison between ANNs and SNNs.

3.1 Experimental Framework

All the code was implemented in Python, using PyTorch¹ as the main framework for implementing and training ANN models, as well as for handling tensors, automatic differentiation, and optimization. The SNN models were implemented with snnTorch², a library built on top of PyTorch that provides spiking neuron models, surrogate gradient training, and spike-based utilities.

The tonic³ library was used to process neuromorphic datasets such as N-MNIST and CIFAR10-DVS. This library provides tools for loading event-based datasets and transforming asynchronous event streams into frame-based temporal representations that can be used by both ANN and SNN models.

The experiments were executed on an Apple Silicon machine (MacBook Pro 2020, Apple M1 chip, 8 GB unified memory). This hardware setup is relevant because it represents a conventional consumer device rather than a specialized neuromorphic processor. Therefore, the experimental results obtained from direct training and energy measurement must be interpreted as results on conventional hardware, not as direct evidence of neuromorphic efficiency.

3.2 Datasets

The MNIST, CIFAR-10, N-MNIST, and CIFAR10-DVS datasets were selected to cover both static and temporal visual data. This distinction is important because SNNs are theoretically better suited to temporal information, while conventional ANNs are highly optimized for dense static data. The motivation for using each of the aforementioned datasets is as follows:

- MNIST⁴ was mainly used as an initial baseline dataset to validate the implementation of ANN and SNN models. MNIST is a standard image classification dataset composed of grayscale handwritten digits from 0 to 9. Each image has a spatial resolution of 28×28 pixels.

¹<https://pytorch.org/>

²<https://snntorch.readthedocs.io/en/latest/>

³<https://tonic.readthedocs.io/en/latest/>

⁴<https://docs.ultralytics.com/es/datasets/classify/mnist>

- CIFAR-10⁵ was used to evaluate whether SNNs could scale from simple grayscale classification to a more challenging static vision task. CIFAR-10 is a static image classification dataset composed of 60,000 RGB images belonging to 10 object classes. Each image has a resolution of 32×32 pixels with three color channels.
- N-MNIST was selected as the first event-based benchmark because it preserves the simplicity of MNIST while introducing temporal neuromorphic information. It is a neuromorphic version of the MNIST dataset generated using a DVS. Instead of static grayscale images, the dataset contains event streams produced by sensor motion over handwritten digits. Each event is characterized by spatial coordinates, timestamps, and a polarity.
- CIFAR10-DVS was the last dataset used. It is an event-based version of CIFAR-10 obtained by recording moving CIFAR-10 images with a DVS. The dataset consists of event streams organized into the same 10 classes as CIFAR-10.

3.3 Considered Models

Two main families of models are implemented: ANNs and SNNs. The ANN models serve as baseline systems, while the SNN models are used to study spike-based temporal computation.

3.3.1 ANN Models

The ANN models are based on CNN architectures. For static datasets such as CIFAR-10, the ANN receives standard image tensors as input. For event-based datasets, temporal event frames are collapsed across the timestep dimension before being passed to the ANN. This allows the ANN to process event-based data as a dense spatial representation. ANN models use convolutional layers for feature extraction, followed by fully connected layers for classification.

These ANN baselines are important because they provide a direct comparison against SNNs on the same datasets. In the case of event-based datasets, the ANN baseline also helps evaluate whether temporal neuromorphic data necessarily favors SNNs or whether conventional models can still exploit aggregated event information effectively.

3.3.2 SNN Models

The SNN models are implemented using LIF neurons. When the membrane potential exceeds a threshold, the neuron emits a spike and resets its state.

The SNN architecture processes inputs over multiple timesteps. At each timestep, spikes are propagated through convolutional and fully connected layers, while neuron states are maintained across time. The final prediction is obtained by accumulating output spikes across all timesteps and selecting the class with the highest spike count.

For visual classification tasks, convolutional SNNs are implemented. These models follow the same general principle as CNNs, using convolutional layers to extract spatial features. However, after each convolutional operation, spiking neuron layers transform continuous membrane potentials into binary spike events. The operations included in SNNs are: convolutional feature extraction layers, spiking LIF neuron layers, pooling operations, fully connected layers, and output spike accumulation.

⁵<https://docs.ultralytics.com/es/datasets/classify/cifar10>

3.4 Information Encoding and Decoding

SNNs process information through discrete spike events distributed over time. Consequently, input data must be transformed into spike trains before being presented to the network, and the resulting spike activity must be later decoded to obtain a final prediction. The choice of encoding strategy determines how information is distributed across timesteps and how efficiently neural activity is generated. Several encoding approaches were explored throughout this work to evaluate their impact on learning dynamics, spike sparsity, and classification performance (see Fig. 3.1).

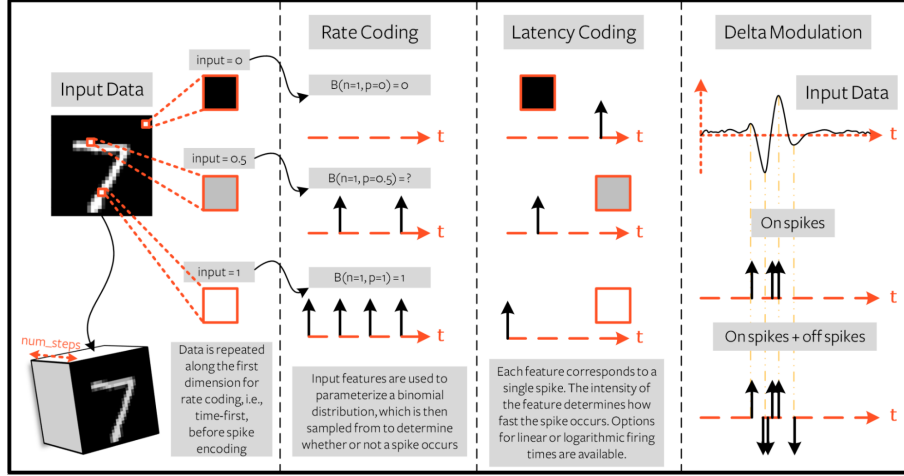


Figure 3.1: SNN encoding methods. *Source: <https://snntorch.readthedocs.io/en/latest/index.html>*

3.4.1 Rate Encoding

Rate encoding was the primary input representation method used throughout this work. In this approach, the intensity of an input value is converted into a probability of spike generation at each timestep. Given an input value $x \in [0, 1]$, the probability of generating a spike at timestep t is defined as $P(S(t) = 1) = x$, where $S(t)$ represents the spike state of a neuron at timestep t . Under this formulation, larger input intensities produce higher firing probabilities, while lower intensities generate fewer spikes. Over multiple timesteps, the average firing rate approximates the original input intensity.

Rate encoding offers several advantages. First, it is simple to implement and closely resembles biological rate coding mechanisms observed in sensory systems. Second, it enables the use of conventional image datasets by transforming static pixel intensities into temporal spike sequences. Finally, it provides a relatively stable representation that facilitates training through surrogate gradient methods. A limitation of rate encoding is that information is distributed across multiple timesteps, requiring repeated sampling of spike activity to accurately represent the original input. As a result, inference latency and computational cost may increase as the number of timesteps grows.

3.4.2 Latency Encoding

Latency encoding represents information through the timing of spike generation rather than through firing frequency. Stronger input stimuli generate spikes earlier in time, while weaker stimuli generate spikes later. Conceptually, neurons receiving larger input values reach their firing threshold more rapidly, producing an inverse relationship between input intensity and spike latency.

Unlike rate encoding, where information is distributed across many spikes, latency encoding attempts to represent information using a small number of highly informative events. Although latency encoding was considered during the design phase of this work, the final experiments primarily relied on rate-based representations due to their simplicity and greater training stability.

3.4.3 Gradient Encoding

Gradient encoding was investigated as an alternative mechanism for emphasizing spatial variations within static images before spike generation. Instead of directly encoding raw pixel intensities, local intensity differences are first extracted through image gradient operations. Regions containing strong transitions, edges, or object boundaries receive greater emphasis during spike generation.

The motivation behind this approach is that edges in images often contain a large proportion of the information required for visual classification tasks. By allocating more spike activity to regions with stronger gradients, the encoding process may increase the information content carried by each spike. This strategy was explored as a method for improving information efficiency and reducing the generation of spikes associated with visually uninformative regions.

3.4.4 Center-Weighted Encoding

Center-weighted encoding was introduced as an experimental strategy specifically motivated by the spatial characteristics of certain images. In datasets such as MNIST, the most relevant visual information is typically concentrated near the center of the image, while peripheral regions contain mostly background pixels. Under standard rate encoding, all pixels contribute equally to spike generation regardless of their spatial location. Center-weighted encoding modifies this behavior by assigning higher spike-generation probabilities to pixels located near the image center. The objective of center-weighted encoding was to investigate whether incorporating prior spatial knowledge into the encoding process could improve learning efficiency and classification performance.

3.4.5 Spike Probability Scaling

Spike probability scaling controls the density of neural activity generated during encoding. Under standard rate encoding, spike probabilities are directly proportional to input intensity values. In spike probability scaling, these probabilities are multiplied by a scaling factor before spike generation: $P(S(t) = 1) = \alpha x$, where α is a scaling coefficient controlling overall spike activity. Increasing the scaling factor produces denser spike trains and higher firing rates, potentially facilitating information transmission. Conversely, lower scaling values promote sparser activity, reducing the number of generated spikes and potentially improving computational efficiency.

This technique was used to investigate the relationship between spike density, classification performance, and network sparsity. The resulting experiments provided insights into the trade-off between information availability and computational cost within spike-based systems.

3.4.6 Output Decoding

Because SNNs produce spike trains rather than continuous-valued outputs, a decoding mechanism is required to transform spike activity into a final prediction. The decoding strategy adopted throughout this work was spike-count decoding. For each output neuron k , the total number of emitted spikes during the simulation window is accumulated according to

$$y_k = \sum_{t=1}^T S_k(t), \quad (3.1)$$

where $S_k(t)$ denotes the spike generated by output neuron k at timestep t and T represents the total number of simulation timesteps. The predicted class is then obtained as $\hat{y} = \arg \max_k y_k$, meaning that the output neuron with the highest accumulated spike count determines the final prediction.

Spike-count decoding is one of the most widely used decoding strategies because it provides a simple and robust approximation of the continuous activations commonly used in conventional neural networks. Alternative decoding methods exist, including latency decoding, where predictions are determined by

the neuron that fires first. Although such approaches can potentially reduce inference latency, they were not considered in the final experimental evaluation of this work.

3.5 Training Methodology

3.5.1 Loss Functions

Two loss functions were mainly considered: (1) Mean Squared Error (MSE) and (2) Cross-Entropy (CE) loss. For ANN models, CE loss was used because it suits well for multi-class classification problems and directly optimizes class probability separation. For SNN models, MSE was frequently used with spike-count targets. In this case, output spikes were accumulated over time and compared with one-hot target vectors. The objective encouraged the correct output neuron to produce higher spike activity than the remaining classes. CE loss was also tested in some SNN experiments to evaluate whether a conventional classification loss could improve optimization. However, MSE was generally more aligned with the spike-count interpretation of SNN outputs.

3.5.2 Reproducibility and Seed Experiments

To improve the statistical reliability of the results, the best performing models were repeated using multiple random seeds, allowing the computation of mean accuracy and standard deviation across independent runs. Using multiple seeds is important because neural network training involves random initialization, random batching, and stochastic optimization, so reporting mean and standard deviation provides a more reliable estimate of model behavior than a single training run.

3.5.3 Data Splitting and Evaluation Protocol

Evaluation was performed using the official train-test partitions provided with each dataset. Model development was performed exclusively on the training data, ensuring complete separation between training and evaluation samples. Consequently, no data leakage occurred between training and testing stages.

3.6 Evaluation Metrics

The evaluation methodology combined predictive, spike-based, and computational metrics. This was necessary because classification accuracy alone does not fully describe the behavior of SNNs.

3.6.1 Performance Metrics

The main performance metric was classification accuracy, defined as the percentage of correctly classified samples in the test set. Training loss was also recorded at each epoch in order to analyze convergence behavior.

For SNN models, spike activity metrics were computed. Total spikes measured the total number of output spikes produced by the model during evaluation. Firing rate was computed as the ratio between emitted spikes and the total number of possible spike events. Sparsity was defined as: $sparsity = 1 - firingRate$. These metrics were used to analyze how active or sparse each SNN model was. They also provided evidence for the theoretical computational advantages of SNNs under event-driven hardware assumptions.

3.6.2 Computational Metrics

Training time and epoch duration were recorded for all relevant experiments. These metrics helped compare the practical computational cost of ANN and SNN training on conventional hardware.

CarbonTracker⁶ was initially tested to estimate energy consumption and CO_2 emissions, but it did not provide meaningful values on the Apple Silicon environment. For this reason, CodeCarbon⁷ was later used as the main energy measurement tool. CodeCarbon provided estimates of execution duration, energy consumed and emissions based on CPU, GPU and RAM utilization.

In addition to measured energy consumption, a theoretical operation-count analysis was performed. ANN computation was estimated using MAC operations, while SNN computation was estimated using spike-triggered SOPs. The SNN operation estimate considered both firing rate and number of timesteps:

$$SNN_{SOPs} \approx ANN_{MACs} \times T \times firingRate, \quad (3.2)$$

where T is the number of timesteps. This analysis was used to estimate the potential computational advantage of SNNs under neuromorphic hardware assumptions, where inactive neurons do not trigger computation and spike-based operations are cheaper than dense MAC operations.

⁶<https://github.com/saintslab/carbontracker>

⁷<https://github.com/mlco2/codecarbon>

Chapter 4

PERFORMANCE EVALUATION

4.1 Static Dataset Experiments

The first stage of experimentation focuses on static image classification datasets. These experiments serve two purposes: to provide a controlled environment for validating the proposed SNN architectures, and to enable the investigation of fundamental SNN properties, such as spike encoding, sparsity and temporal dynamics, before introducing the additional complexity of event-based data.

4.1.1 MNIST

In the following experiments, the ANN model consists of a fully connected architecture with ReLU activation in the hidden layer. During the forward pass, the ANN processes the input data instantaneously and computes the CE error based on these continuous activations. The SNN counterpart replaces the ReLU activations with LIF neurons while maintaining the same overall topology. At the input, the MNIST images are first converted into spike trains using rate encoding, while the loss function is the same as that used by the ANN, i.e., CE loss.

Experiment #1A: ANN vs SNN Baseline Comparison

The first experiment consists of a direct comparison between a conventional ANN and an SNN using equivalent Multi-Layer Perceptron architectures on MNIST. The objective of this experiment is not to maximize performance but rather to verify that the implemented SNN is capable of learning successfully through surrogate gradient training. Figure 4.1 shows the accuracy and time comparison.

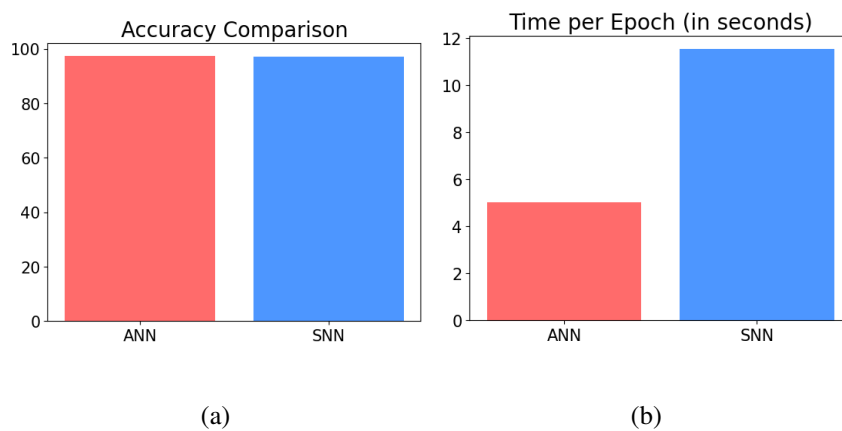


Figure 4.1: ANN and SNN accuracy and time per epoch comparison.

The results show that both models achieve comparable classification accuracy, reaching approxi-

mately 96–97% on the MNIST test set. Although predictive performance is similar, the SNN requires significantly longer training times due to the simulation of temporal dynamics across multiple timesteps on conventional hardware.

To better understand the internal behavior of the SNN, a spike activity analysis is implemented. Analyzing the quantity and distribution of spikes provides valuable information regarding network efficiency and neural activity. This analysis is repeated for different numbers of simulation timesteps to investigate how temporal resolution influences spike activity and network behavior. Below, Figure 4.2 shows the obtained results for total spikes, sparsity, accuracy and firing rate across multiple timesteps.

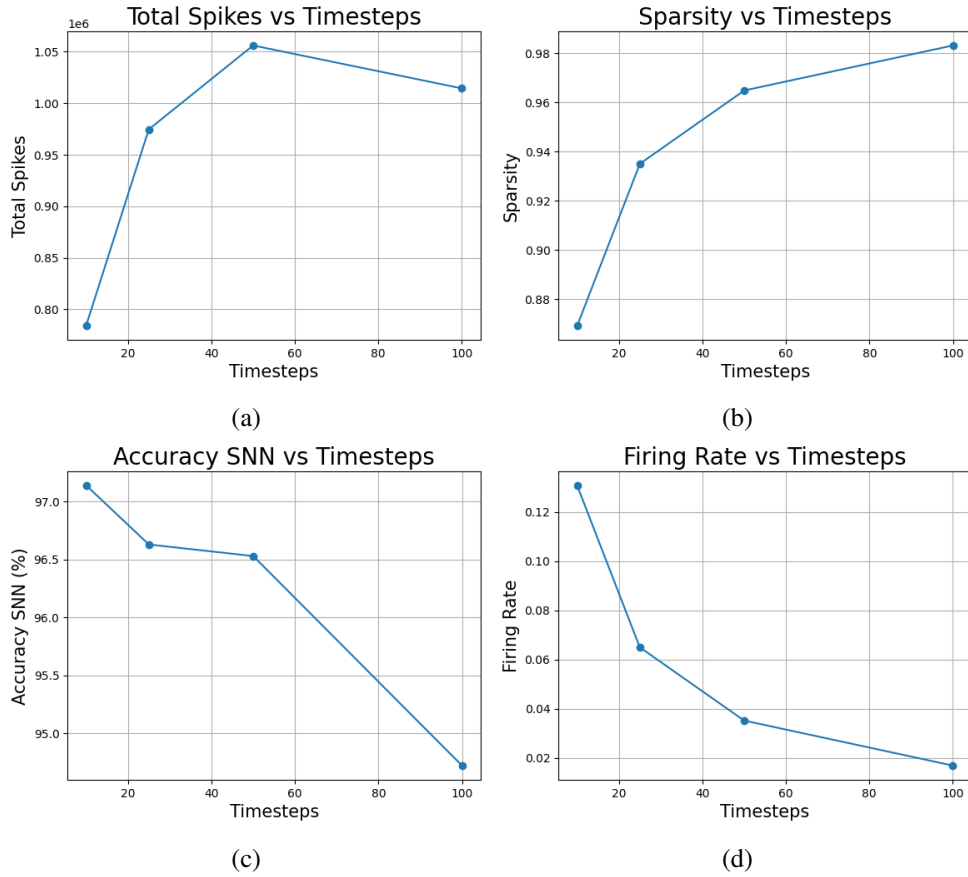


Figure 4.2: Spike analysis from the first SNN model.

The results show that increasing the number of timesteps tends to produce a higher total number of spikes. Since the network is simulated for a longer temporal window, neurons have more opportunities to accumulate membrane potential and emit spikes. Even if individual neurons become less active at each timestep, the larger simulation horizon generally results in a greater absolute number of emitted spikes. In addition, higher timestep values produce lower firing rates. Although more spikes are generated overall, the number of possible spike events grows proportionally with the number of timesteps. This entails that the increase in spike count is smaller than the increase in available temporal positions, causing the average firing rate to decrease. As a direct consequence, sparsity increases with the number of timesteps. A lower firing rate means that a greater proportion of neuron states remain inactive throughout the simulation. Therefore, extending the temporal window naturally leads to sparser neural activity.

Conclusions of Experiment #1A: The baseline experiment successfully validates the implemented SNN, but also highlights several limitations and motivates the next stages of experimentation:

- First, the similar performance achieved by ANN and SNN models suggests that MNIST might be too simple to reveal meaningful differences between both paradigms. This observation motivates the transition to more challenging datasets, particularly CIFAR-10.

- On top of that, the spatial structure of MNIST inspires the exploration of alternative encoding strategies. Since most relevant information is concentrated near the center of the image and around digit contours, center-weighted and gradient-based encoding methods need to be investigated to evaluate whether more informative spike representations could improve learning.
- Moreover, the relationship between spike activity and classification performance motivates a systematic study of spike probability scaling to determine whether controlling input spike density could improve the balance between information transmission and sparsity.
- The experiments also reveal an interesting relationship between temporal resolution and network performance. Surprisingly, the highest accuracies were obtained using a small number of timesteps, as seen in Figure 4.2c. Since MNIST contains no intrinsic temporal information, additional timesteps mainly introduce redundancy rather than useful information.

Taken together, these observations demonstrate that the baseline MNIST experiment served not only as an implementation validation step, but also as a source of research questions that guided the subsequent development of the project: a spike probability scaling experiment, the implementation of the center-weighted and gradient-based encoding strategies and an evaluation on a more complex dataset, in this case CIFAR-10.

Experiment #1B: Spike Probability Scaling

This first encoding-related experiment focuses on the spike probability parameter used in rate encoding. The objective is to evaluate how different spiking probabilities influence the model’s accuracy and to understand the underlying spiking behavior to make it predictable and controllable. This parameter effectively controls how much activity enters the network. A wide range of scaling values between 0.1 and 1.5 are evaluated.

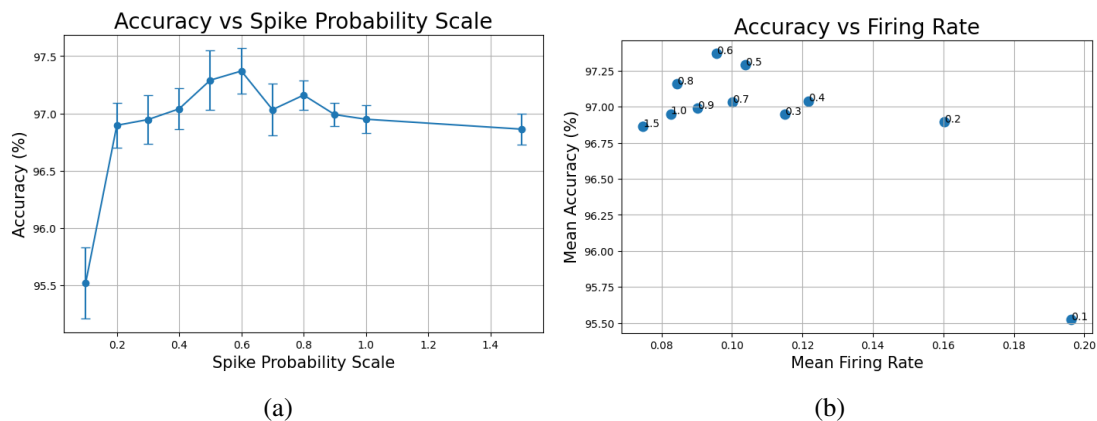


Figure 4.3: Effect of different spike probabilities on accuracy and firing rate.

The experiments reveal that intermediate values consistently produce the best classification performance, with optimal results appearing around 0.6 (Figure 4.3a). A particularly interesting observation emerges when analyzing the relationship between spike probability scaling and network activity. Although the scaling factor was introduced to increase the probability of generating input spikes, the resulting firing activity within the network evolves in the opposite direction. Higher scaling values consistently produce lower firing rates and higher sparsity levels.

This behavior can be explained by considering how the network adapts its synaptic weights during training. The spike probability scaling parameter does not directly modify the firing threshold of neurons or their intrinsic dynamics. Instead, it changes the density of the spike-based input representation presented to the network. When low scaling values are used, the encoded input becomes highly sparse

and contains relatively little information. Larger weights amplify the effect of each incoming spike, allowing neurons to reach the firing threshold more easily, so even isolated input spikes can trigger large membrane potential variations, producing widespread and often redundant spike activity throughout the network. In contrast, when higher scaling values are used, the input representation already contains a large amount of spike activity. Neurons become more discriminative, and only those responding to meaningful patterns accumulate sufficient membrane potential to generate output spikes. As a result, overall firing activity decreases, and network sparsity increases.

Conclusions of Experiment #1B:

- Sparsity in SNNs is not determined exclusively by the encoding strategy but also emerges as a consequence of the interaction between input activity, synaptic adaptation and temporal integration dynamics. The highest classification accuracy was achieved precisely within the region where this balance produced sparse but informative neural activity, indicating that efficient spike-based computation depends on controlling both the quantity and the quality of the generated spikes.

Experiment #1C: Center-Weighted Encoding

Although it seems from previous experiments that the performance cap for the MNIST dataset is minimal, it is worth trying different encoding mechanisms to further evaluate performance and spiking behavior. Accordingly, we now use a center-weighted encoding strategy to attempt to incorporate spatial prior knowledge. Since MNIST digits are naturally centered within the image, a Gaussian weighting mask is applied before spike generation. This mask increases the contribution of central pixels while reducing the influence of peripheral regions.

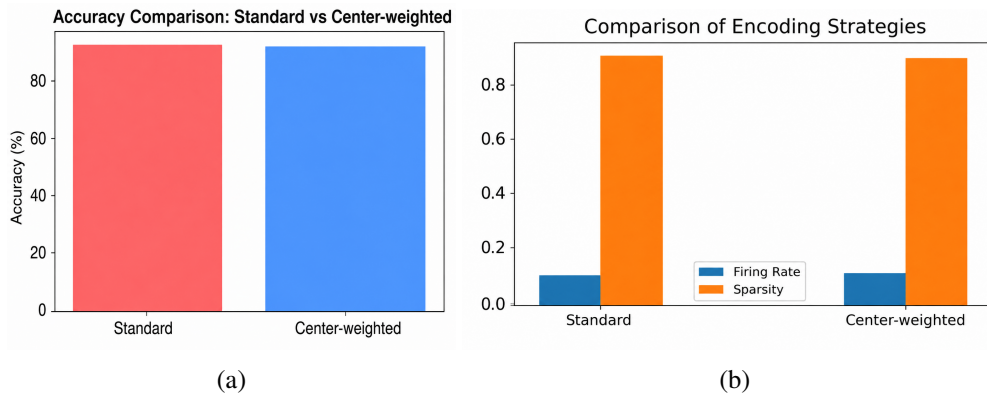


Figure 4.4: Center-Weighted encoding accuracy and spike activity results.

The results are almost identical to the standard encoding. Classification accuracy remain close to the baseline values (Figure 4.4a), and no significant efficiency improvements are observed.

Conclusions of Experiment #1C:

- These findings suggest that the network is already capable of identifying relevant spatial information without requiring explicit weighting mechanisms.

Experiment #1D: Gradient-Based Encoding

The final MNIST encoding experiment investigates whether emphasizing image contours could improve learning. Gradient maps are computed using Sobel filters, which act as a geometric feature extractor by removing redundant background information and concentrating mathematical energy on object boundaries, optimizing learning performance for SNNs Eshraghian et al. (2023). This, combined with the

original images before spike generation, results in a representation that highlights edges and local intensity variations. This approach is motivated by the observation that object boundaries often contain highly discriminative information.

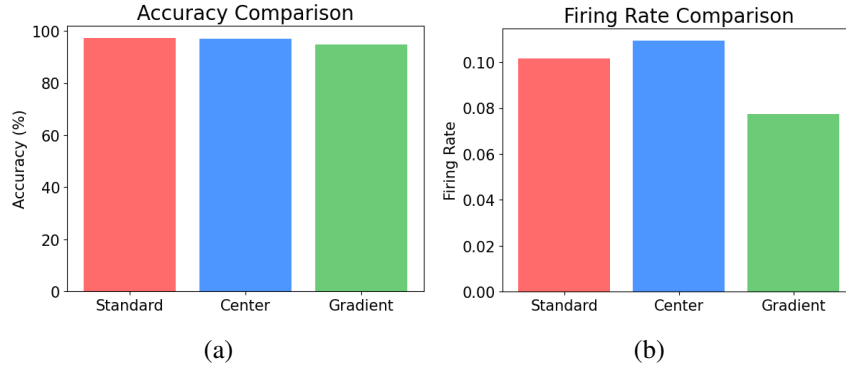


Figure 4.5: Accuracy and firing rate comparison between all encoding strategies

Despite producing visually meaningful spike patterns, gradient-based encoding results in a slight reduction in classification accuracy, as observed in Figure 4.5a. The most likely explanation is that MNIST classification relies not only on contour information but also on internal intensity distributions. By emphasizing edges, part of the useful information contained within the digit structures may have been weakened. The spike activity analysis reveals clear differences between the encoding strategies (Figure 4.5b). Center-weighted encoding produces the highest firing rate and lowest sparsity by concentrating activity in the most informative image regions. In contrast, gradient encoding generates the sparsest representation because spike activity was restricted primarily to edges and contours.

Conclusions of Experiment #1D:

- Despite these differences in neural activity, the impact on classification performance remains limited. Therefore, the results suggest that modifying the spatial distribution of spikes alone is insufficient to overcome the limitations imposed by static image encoding.
- While encoding influences activity patterns and computational efficiency, it does not fundamentally solve the mismatch between static visual information and temporal spike-based processing.

4.1.2 CIFAR-10

After completing the MNIST experiments, the study moved towards CIFAR-10. This dataset represents a substantially more challenging benchmark for SNNs. Consequently, the transition also requires moving from simple MLP architectures to CNNs capable of extracting hierarchical spatial features.

Experiment #2A: First ANN vs SNN Comparison

The initial CIFAR-10 comparison was performed using equivalent ANN and SNN convolutional architectures. The ANN baseline consisted of two convolutional blocks followed by a fully connected classifier. The corresponding SNN replaced the conventional activations with LIF neurons while preserving the same overall architecture.

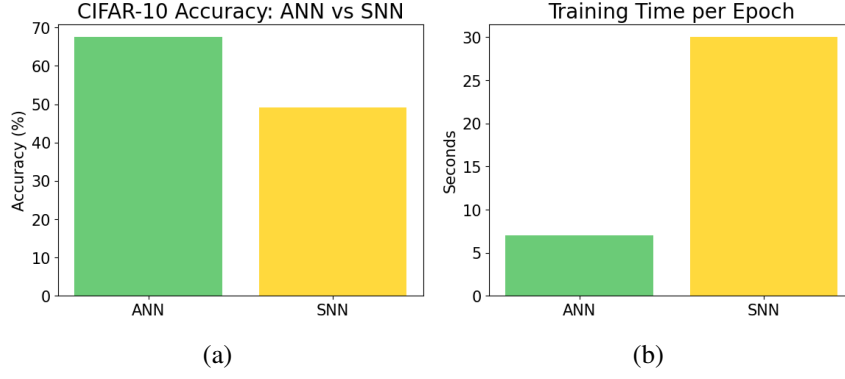


Figure 4.6: CIFAR-10 accuracy and time comparison.

The results reveal a substantial performance gap between the two approaches. While the ANN achieves approximately 67.5% accuracy, the initial SNN reaches only around 49% (Figure: 4.6a). Moreover, SNNs are significantly slower due to temporal processing (Figure 4.6b). Unlike MNIST, CIFAR-10 contains richer visual information that is more difficult to preserve during spike encoding.

Conclusions of Experiment #2A:

- These results show that several changes are needed in the whole SNN structure approach in order to exploit their full potential.

It must be said that center-weighted and gradient encoding were also evaluated on CIFAR-10. However, the observed behavior was consistent with the trends previously identified on MNIST, producing only minor variations in spike activity and no significant improvement in classification performance.

Experiment #2B: Loss Function Comparison

One of the first modifications investigated is the loss function. Initially, CE loss was used due to its widespread adoption in conventional image classification. However, previous literature suggests that MSE may provide smoother optimization dynamics in SNNs (Neftci et al., 2019; Eshraghian et al., 2023). Indeed, the experiment comparing both losses reveals a consistent advantage for MSE, as seen in Figure 4.7.

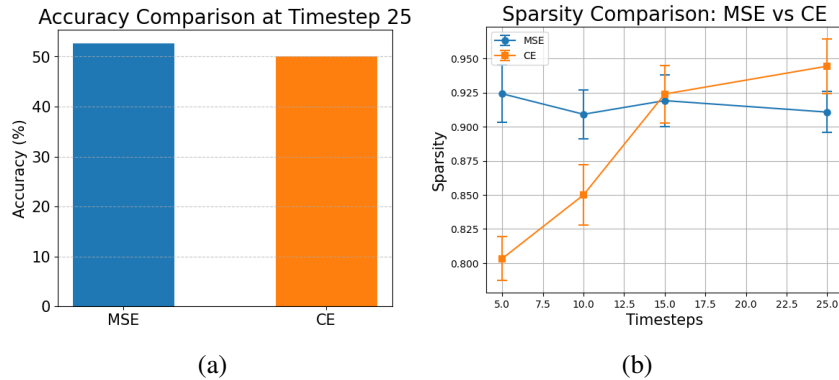


Figure 4.7: Accuracy and sparsity on MSE and CE losses comparison.

This experiment is performed on various timesteps, but timestep 25 shows the best results for both. The MSE-based model achieves higher accuracy while simultaneously producing more stable learning patterns. This improvement is likely related to the smoother error signals generated by MSE, which better complement surrogate-gradient learning.

An interesting difference emerged when analyzing spike activity. CE tended to generate higher firing rates at low timesteps but substantially lower activity at larger simulation windows (Figure 4.7b). One possible explanation is that the stronger gradients produced by CE encourage rapid early activation, leading neurons to reach firing thresholds quickly and concentrate activity at the beginning of the temporal sequence. In contrast, MSE generates smoother error signals that promote a more balanced distribution of spike activity across time. This behavior likely contributes to the more stable optimization process and improves classification accuracy throughout the experiments.

Conclusions of Experiment #2B:

- MSE loss consistently outperforms CE loss on CIFAR-10, achieving higher classification accuracy and more stable learning behavior across different simulation lengths.
- Consequently, MSE was selected as the preferred loss function for the subsequent experiments.

Experiment #2C: tdbN-Inspired Normalization

One of the most significant improvements was obtained through the introduction of a normalization mechanism inspired by Threshold-Dependent Batch Normalization (Zheng et al., 2020). This tdbN-inspired mechanism acts like a biological voltage regulator, ensuring pre-synaptic inputs are perfectly calibrated against the neural firing threshold at every single moment in time.

The implemented SNNNorm layer normalizes activations before spike generation, helping stabilize membrane potentials and spike distributions. This modification produces a substantial increase in classification accuracy and noticeably improves training stability.

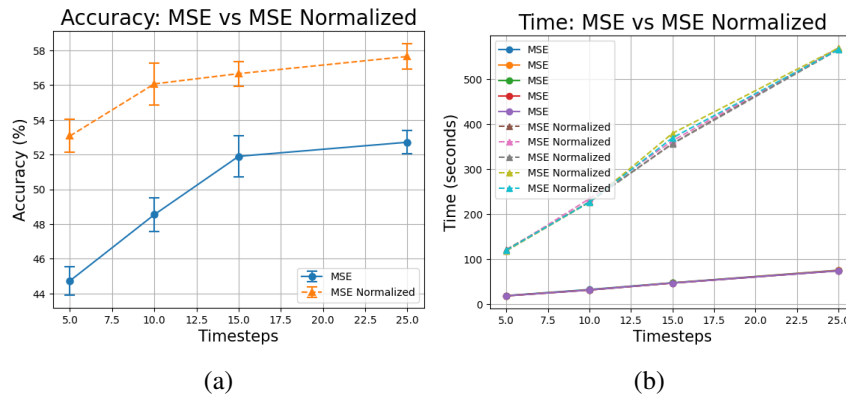


Figure 4.8: Accuracy and time comparison between the model with and without normalization.

As we see in Figures 4.8a and 4.8b, the introduction of tdbN produces one of the most significant improvements. The model achieves a significantly higher classification accuracy, although training time increases due to the additional normalization operations performed at each timestep.

By normalizing activations at each timestep, tdbN maintains membrane potentials within a more controlled operating range. As a consequence, neurons are more likely to operate near their firing threshold rather than becoming permanently inactive or excessively active. This effect is reflected in the spike analysis, where the firing rate increases, and sparsity decreases slightly compared to the baseline model. Although lower sparsity is often associated with increased computational activity, in this case, the change appears beneficial because the original network operates in an extremely sparse regime. The additional spike activity enables more neurons to participate in information processing and gradient propagation during training.

Conclusions of Experiment #2B:

- These results suggest that the primary benefit of tdbN is not simply increasing neural activity, but regulating it.

Experiment #2D: Kaiming Initialization

To keep improving performance, weight initialization seems important and is also investigated through Kaiming initialization. The objective is to improve signal propagation during the early stages of training, as this method ensures that the variance of inputs to the membrane potential remains stable across layers at time step zero. This provides a reliable baseline signal before the internal recurrent temporal dynamics take over, preventing early training collapse.

Although the initialization strategy produced slightly more stable convergence, the overall impact on classification accuracy slightly decreased, as seen in Figure 4.9.

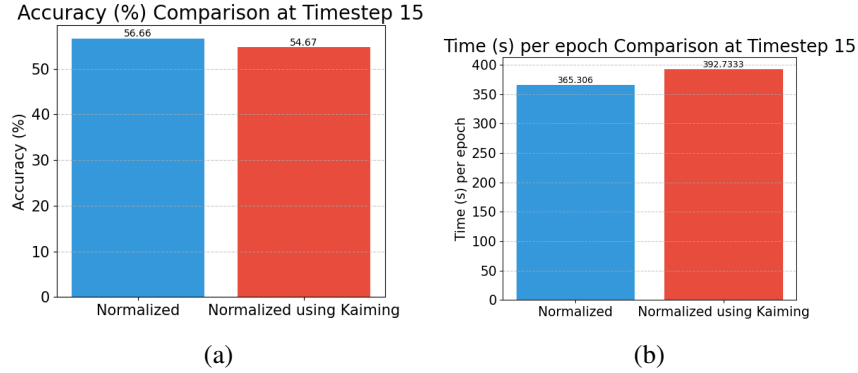


Figure 4.9: Kaiming accuracy and time comparison at timestep 15.

Conclusions of Experiment #2D:

- These results suggest that conventional Kaiming initialization, originally designed for ReLU-based ANNs, may not fully capture the specific dynamics of spiking neurons.

Motivated by recent research, threshold-dependent and leak-aware initialization schemes specifically designed for LIF neurons were also considered. These approaches incorporate firing thresholds and membrane dynamics directly into the weight scaling process. However, preliminary experiments did not produce meaningful improvements over standard initialization methods and were therefore not explored further.

Experiment #2E: Deeper Convolutional Architectures

Another CIFAR-10 improvement focused on increasing model capacity. Additional convolutional layers and wider feature representations are introduced. The improved architecture increases classification performance slightly but also produces a considerable increase in training time, as seen in Figure 4.10.

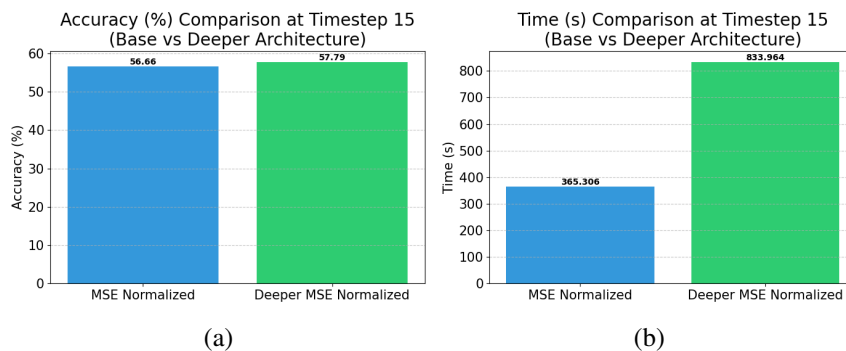


Figure 4.10: Deeper CNN accuracy and time comparison at timestep 15.

Conclusions of Experiment #2E:

- This experiment demonstrates that increasing network complexity alone is insufficient to overcome the limitations introduced by spike-based representations of static images, suggesting that the primary bottleneck lies not in model capacity but in the conversion of static information into temporal spike trains.

Experiment #2F: Beta Analysis on CIFAR-10

Finally, the membrane decay parameter β is investigated. Tuning this parameter seems interesting as it adjusts the memory window, essentially dictating how long a neuron “remembers” its past inputs. Theoretically, larger beta values allow neurons to retain information for longer periods of time, increasing temporal memory. Experiments reveal that lower beta values generally produce better results on CIFAR-10 (Figure 4.11). Since CIFAR-10 contains static images with no intrinsic temporal structure, long-term temporal accumulation provides limited benefits. Instead, more reactive neurons with shorter memory appeared more suitable for processing static visual information.

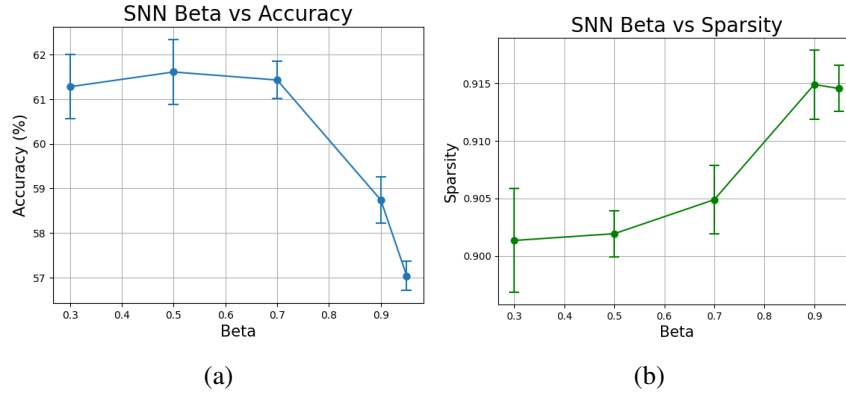


Figure 4.11: Accuracy and sparsity comparison between different β values.

Conclusions of Experiment #2F:

- These results suggest that the optimal temporal dynamics of an SNN should be matched to the characteristics of the dataset.

4.1.3 Main Findings on Static Dataset Experimentations

The experiments conducted in MNIST and CIFAR-10 provide valuable insight into the behavior of SNNs when applied to static visual data:

- A particularly relevant observation concerns computational efficiency. Several modifications are capable of producing modest accuracy improvements, but these gains are frequently followed by substantially longer training times.
- Furthermore, input encoding represent a major bottleneck for SNN performance on static datasets. Rate encoding distributes information across multiple timesteps, creating temporal redundancy and stochastic noise, while alternative approaches, such as center-weighted and gradient encoding, attempt to emphasize specific spatial regions of the image. However, despite producing different spike distributions and activity patterns, none of the evaluated encoding strategies generate a substantial improvement in classification accuracy. These suggest that the primary limitation is not how static information is encoded, but rather the fact that static images must be artificially transformed into temporal signals in the first place.

For these reasons, the next stage of the study focuses on temporal neuromorphic datasets, where information is inherently represented through events distributed over time.

4.2 Temporal Dataset Experiments

Following the experiments on static image datasets, the focus shifts towards temporal and event-based data. This transition is particularly important because one of the main motivations behind SNNs is their ability to process temporal information naturally through spike-based dynamics.

Unlike static datasets, event-based datasets contain explicit temporal information generated by DVS. Therefore, they represent a more suitable environment for evaluating whether the theoretical advantages of SNNs translate into practical improvements.

4.2.1 N-MNIST

This first temporal dataset provides an interesting opportunity to evaluate whether the transition from static images to temporal event data benefits SNNs relative to conventional ANNs.

Experiment #3A: ANN vs SNN Comparison

The first experiment compares ANN and SNN architectures on N-MNIST. For the ANN baseline, temporal frames are collapsed across time and processed as conventional images. The SNN, in contrast, receives the complete temporal sequence and processes it through LIF neurons over multiple timesteps. Similarly to MNIST results, the performance difference between ANNs and SNNs is considerably small, as seen in Figure 4.12.

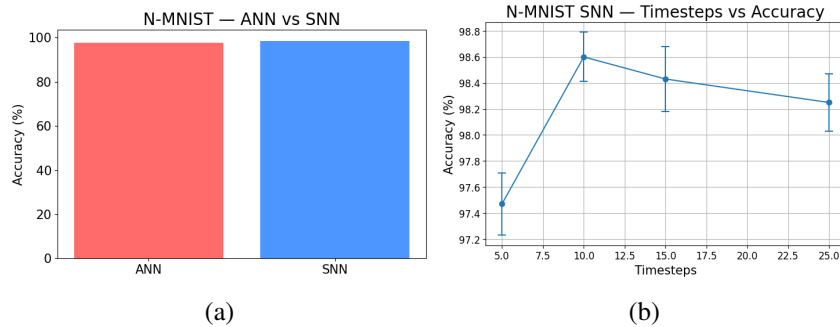


Figure 4.12: ANN and SNN accuracies (a), SNN accuracy with different timesteps (b).

Conclusions of Experiment #3A:

- Both approaches achieve very high classification accuracy, exceeding 98%. The best results for the SNN model appears at 10 timesteps, suggesting that enough temporal information is already captured at that point and additional temporal resolution becomes redundant, similar to what we saw with the MNIST dataset.
- The final SNN achieves a sparsity close to 90%. Despite this relatively low activity level, classification accuracy remains extremely high. This demonstrates that a small proportion of active neurons is sufficient to represent the relevant information contained within the event streams.

4.2.2 CIFAR10-DVS

After completing the N-MNIST experiment, the study moves towards CIFAR10-DVS. CIFAR10-DVS represents a significantly more challenging benchmark because it combines the complexity of CIFAR-10 with the temporal characteristics of event-based sensing.

Experiment #4A: ANN vs SNN Comparison

The first experiment on CIFAR10-DVS employs a relatively simple convolutional SNN. Although the network is capable of learning meaningful representations from the event-based data, its performance remains substantially below that of the corresponding ANN baseline. Similar to the observations made on N-MNIST, the best SNN results are obtained using 10 simulation timesteps.

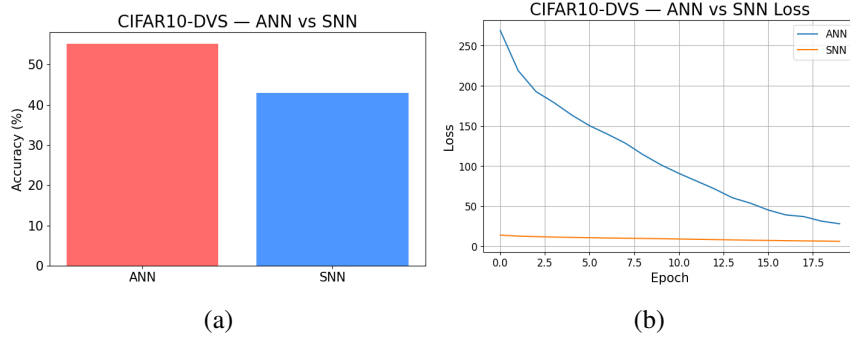


Figure 4.13: Accuracy and loss comparisons on CIFAR10-DVS.

While the ANN achieves approximately 55% classification accuracy, the SNN reaches only around 43% (Figure 4.13a), despite being trained on data that is inherently temporal and event-based.

Conclusions of Experiment #4A:

- These results clearly show that the presence of temporal information alone does not automatically favor spike-based computation. The ability of an SNN to exploit event streams seems to depend strongly on architectural design, preprocessing strategies and temporal feature extraction capabilities.
- The training loss curves provide additional insight into this behavior. The SNN loss reduction is considerably slower than that observed for the ANN (Figure 4.13b). This suggests that the learned representations are not expressive enough to capture the complex temporal-spatial structure of CIFAR10-DVS.

At this stage, improving the network architecture appears to be the most natural direction. Modern state-of-the-art approaches on CIFAR10-DVS frequently employ considerably more sophisticated models, including residual spiking networks, attention mechanisms, recurrent temporal modules, deeper hierarchical architectures and event-native processing pipelines specifically designed for neuromorphic data. However, their introduction would require substantial increases in model complexity, training time and experimental resources, potentially shifting the focus away from the comparative analysis between conventional neural networks and biologically inspired spiking models.

Consequently, to verify whether this limitation is due to insufficient training, additional experiments with a higher number of epochs were tried, but produced only marginal improvements in final accuracy. The persistence of this performance gap indicated that the problem was not mainly related to optimization duration, but to the representational capacity of the architecture itself.

Experiment #4B: Beta Analysis on CIFAR10-DVS

The membrane decay parameter is explored again on CIFAR10-DVS. In this context, temporal memory becomes more relevant because event streams evolve continuously over time.

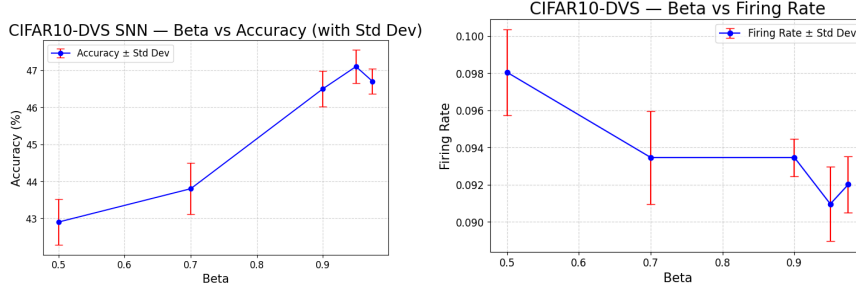


Figure 4.14: Accuracy and firing rate with different β values.

Interestingly, the experiments demonstrated that larger beta values generally produced better performance than those observed on static datasets, in contrast to the results on the CIFAR-10 dataset.

Conclusions of Experiment #4B:

- This behavior is consistent with the theoretical role of membrane integration: longer temporal memory allows neurons to accumulate information across multiple event frames before generating spikes.
- It also aligns with the nature of the data: for event streams, time steps segment the input into distinct sequential frames, while for static images, they simply duplicate the exact same image multiple times.

4.3 Spike Activity Analysis

4.3.1 Sparsity and Firing Rate Trends

Across all experiments, the implemented SNNs operate in highly sparse regimes, as seen in Figures 4.15a and 4.15b. In most cases, firing rates remain below 10%, meaning that more than 90% of all potential spike events never occur. The firing rate is strongly influenced by the selected encoding strategy. Input representations generating excessive spike activity often produce unstable learning behavior and do not necessarily improve classification accuracy. The number of timesteps influence how neural activity is distributed through time. Increasing temporal resolution generally reduces instantaneous firing rates while increasing the total number of emitted spikes. These observations suggest that firing rate is an important indicator of how efficiently information is being represented within the network.

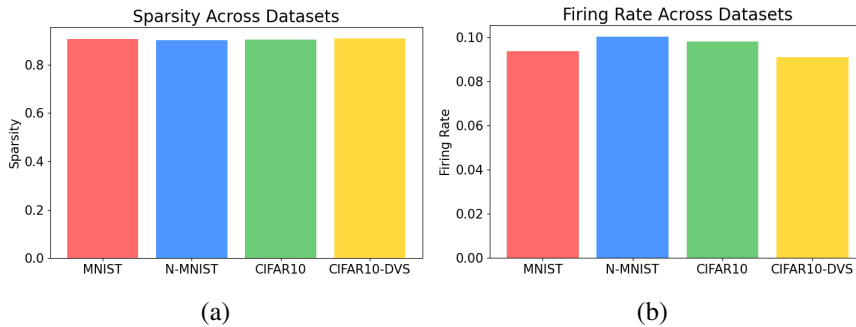


Figure 4.15: Sparsity and firing rate values across all datasets.

4.3.2 Relationship Between Spike Activity and Accuracy

An interesting question investigated during this project was whether higher spike activity leads to better classification performance. The results indicate that this relationship is not linear. Experiments involving

spike probability scaling demonstrate that increasing input activity beyond a certain point often degrades performance. Excessive firing tends to reduce the benefits of temporal integration by making neuron responses overly dense and less discriminative. Conversely, extremely sparse activity can limit information transmission and prevent effective learning. The best-performing models generally operate within an intermediate activity range, suggesting the existence of a balance between information availability and sparsity.

4.3.3 Static vs. Temporal Datasets

The comparison between static and temporal datasets reveals notable differences in spike behavior. For static datasets such as MNIST and CIFAR-10, temporal dynamics are artificially introduced through encoding mechanisms. Consequently, spike activity largely reflects the encoding process itself rather than intrinsic temporal information contained within the data.

The results suggest that temporal datasets allow SNNs to exploit their temporal processing capabilities more naturally than ANNs as the performance gap between them becomes smaller in event-based tasks than in conventional image classification problems. Overall, the spike analysis confirms that SNN behavior is strongly dependent on both the characteristics of the dataset and the mechanisms used to represent information temporally.

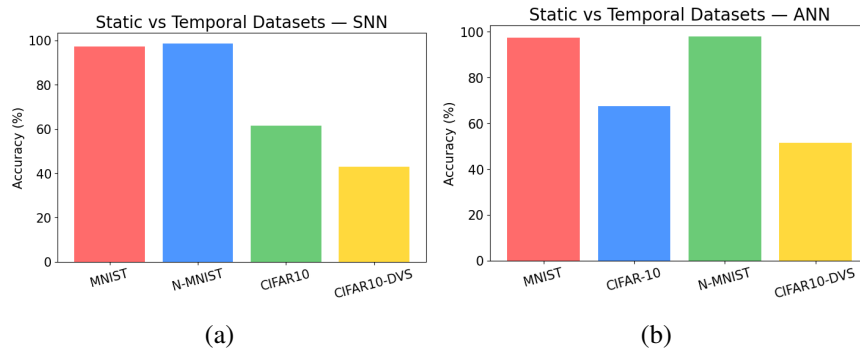


Figure 4.16: Accuracy results across every dataset.

4.4 Energy Analysis and Measurements

In addition to evaluating classification performance and spike activity, an energy analysis is conducted using the CodeCarbon framework. CodeCarbon successfully estimates energy consumption, execution time and associated CO_2 emissions based on CPU, GPU, and memory utilization during training.

The objective of this analysis is not to obtain an exact measurement of electrical consumption, but rather to compare the practical computational cost of ANN and SNN training under identical hardware conditions. CodeCarbon estimates energy consumption by monitoring hardware utilization throughout execution and combining this information with hardware-specific power models. As all experiments were executed on the same machine, these values provide a useful relative comparison between architectures. This energy analysis is exclusively applied to experiments with substantial execution durations where resource tracking yielded meaningful insights.

4.4.1 CodeCarbon Results and Interpretation

The measured energy consumption obtained with CodeCarbon is summarized in Table 4.1.

For both temporal datasets where ANN and SNN models are directly compared, the SNN consistently requires more training time and consumes more energy than the corresponding ANN baseline. On CIFAR10-DVS, the SNN requires approximately 17% more execution time and consumes around 21%

Table 4.1: Performance of ANN (in green) vs. SNN (in blue) in CIFAR10-DVS, N-MNIST, and CIFAR-10. Energy consumption and carbon emissions correspond to the mean values obtained across three independent runs using different random seeds.

Dataset	Mean Duration (s)	Mean Energy (kWh)	CO ₂ Emissions (kg CO ₂ eq)
CIFAR10-DVS	848.7 / 996.8	0.00156 / 0.00189	0.000271 / 0.000329
N-MNIST	2151.6 / 2913.7	0.00397 / 0.00537	0.000691 / 0.000934
CIFAR-10	4256.4	0.00911	0.00159

more energy. Similarly, on N-MNIST, the SNN consumes approximately 35% more energy while also requiring substantially longer execution times.

At first glance, these results may appear to contradict the most frequently cited advantage of SNNs: improved computational efficiency. However, this contradiction can be explained by the hardware platform on which the experiments were executed. All experiments are performed on conventional von Neumann hardware using dense tensor operations implemented through PyTorch and snnTorch. Under this execution model, SNNs cannot fully exploit their event-driven nature. Although spike activity is sparse, the simulator must still update membrane potentials, evaluate thresholds and process every neuron at every timestep. Consequently, computation is performed even when no spikes occur. Furthermore, SNNs require temporal simulation across multiple timesteps, effectively repeating network evaluation several times for each input sample.

As the CodeCarbon measurements represent measurements of software execution cost rather than measurements of intrinsic spike-based efficiency, a complementary operation-count analysis is performed. This analysis estimates the number of MAC operations required by ANN models and compares them with the number of spike-triggered SOPs required by SNNs under neuromorphic assumptions. This methodology attempts to estimate the computational behavior that would be observed on specialized event-driven hardware.

4.4.2 Operation-Count Analysis

Instead of measuring hardware utilization directly, this methodology estimates the number of elementary arithmetic operations performed during inference and uses established energy models from the neuromorphic computing literature to approximate the theoretical energy cost of ANN and SNN computation.

MACs and SOPs

Conventional ANNs primarily rely on MAC operations. Each neuron computes a weighted sum of its inputs, requiring both multiplication and addition operations. Because every connection is evaluated during inference, ANN computation is generally dense. In contrast, SNNs rely on SOPs. When a neuron does not emit a spike, no information is transmitted and no synaptic operation occurs. Computation is therefore event-driven and depends directly on spike activity. The operation-based analysis assumes that ANN inference requires a fixed number of MACs determined by the network architecture, while SNN inference requires only the subset of synaptic operations activated by spikes. Following commonly adopted estimates in neuromorphic computing literature, the energy cost of a MAC operation is assumed to be approximately $3.1pJ$, whereas the cost of a spike-triggered SOP is assumed to be approximately $0.9pJ$ (Zheng et al., 2020). These values do not represent the specific hardware used in this thesis but provide a standardized basis for comparing computational paradigms.

Estimation Methodology

Unlike ANNs, SNNs perform computation across multiple simulation timesteps. Consequently, the total number of potential synaptic operations depends on both network size and simulation duration. The total number of active synaptic operations was estimated as: $SOP_{active} \approx MAC_{ANN} \times T \times firingRate$

where MAC_{ANN} represents the equivalent ANN operation count. The corresponding theoretical energy consumption was then estimated as: $E_{ANN} \approx MAC_{ANN} \times E_{MAC}$ and $E_{SNN} \approx SOP_{active} \times E_{SOP}$ where E_{MAC} and E_{SOP} denote the energy cost of individual MAC and SOP operations respectively.

Operation-Count Results

The results obtained from this analysis are summarized in Table 4.2. As shown, the temporal nature of SNN computation introduces an important trade-off. Although sparse activity reduces the number of active synaptic operations, the network must still be simulated across multiple timesteps. Consequently, increasing the number of timesteps partially offsets the efficiency gained through sparsity. The event-based datasets produce a different behavior. Both N-MNIST and CIFAR10-DVS require fewer timesteps while maintaining similar firing rates. Consequently, the temporal overhead is reduced and the theoretical efficiency advantage increases substantially.

The operation-based analysis explains why the CodeCarbon measurements and the theoretical efficiency estimates appear to disagree. CodeCarbon measures execution on conventional hardware, where sparse spike activity cannot eliminate computation. The operation analysis instead estimates behavior under neuromorphic assumptions, where inactive neurons do not consume computational resources. Together, both analyses provide complementary perspectives on SNN efficiency and highlight the critical role of hardware architecture in determining the practical advantages of spike-based neural computation.

Table 4.2: SNN metrics including timesteps, firing rate, estimated active Synaptic Operations (SOPs), and energy reduction.

Dataset	Timesteps	Firing Rate	Estimated Active SOPs	Energy Reduction
CIFAR-10	15	0.0981	14.71 M	$\times 2.34$
N-MNIST	10	0.1004	2.01 M	$\times 3.43$
CIFAR10-DVS	10	0.0910	9.10 M	$\times 3.79$

4.5 Summary of Experiments

The Table 4.3 summarizes the experiments done as well as the main findings and implications.

Table 4.3: Summary of the experimental findings and their implications.

Experiment	Main Finding	Implication
Experiment #1A: MNIST ANN vs SNN Baseline	Similar accuracy despite longer training times for SNNs	Validated the implementation and showed that simple datasets are insufficient to reveal major differences between ANN and SNN paradigms.
Experiment #1B: Spike Probability Scaling	Intermediate spike activity produced the best results	Demonstrated that successful SNN learning requires a balance between information transmission and sparsity.
Experiment #1C: Center-Weighted Encoding	Minimal impact on performance	Suggested that manually emphasizing spatial regions is insufficient to overcome the limitations of static spike encoding.
Experiment #1D: Gradient Encoding	Different spike distributions but no accuracy improvement	Showed that modifying spike placement alone does not solve the mismatch between static images and temporal computation.

Continues on next page...

... Summary of experiments (Continued from previous page)		
Experiment	Main Finding	Implication
Experiment #2A: CIFAR-10 ANN vs SNN	Large performance gap appeared on a more complex dataset	Revealed the limitations of current SNN architectures and static-data encoding methods.
Experiment #2B: Loss Function Comparison	MSE outperformed CrossEntropy	Confirmed that optimization strategies commonly used in ANNs are not always optimal for SNN training.
Experiment #2C: tdBN-inspired Normalization	Largest performance improvement among architectural modifications	Highlighted the importance of stabilizing membrane potentials and spike distributions during training.
Experiment #2D: Kaiming Initialization	Limited impact on final accuracy	Suggested that initialization is less critical than encoding and normalization for SNN performance.
Experiment #2E: Deeper Convolutional Networks	Higher complexity yielded only modest gains	Indicated that increasing model capacity alone does not solve encoding-related limitations.
Experiment #2F: Beta Analysis on CIFAR-10	Lower beta values performed better	Showed that short temporal memory is preferable when processing static visual information.
Experiment #3A: N-MNIST ANN vs SNN	Comparable performance with high sparsity	Provided evidence that SNNs can effectively exploit naturally temporal event-based data.
Experiment #4A: CIFAR10-DVS ANN vs SNN	ANN remained superior despite temporal input	Demonstrated that temporal information alone does not guarantee SNN superiority.
Experiment #4B: Beta Analysis on CIFAR10-DVS	Higher beta values improved performance	Confirmed that longer temporal memory becomes beneficial when genuine temporal information exists.
CodeCarbon Analysis	SNNs consumed more time and energy on conventional hardware	Showed that practical efficiency depends strongly on the execution platform.
Operation-Count Analysis	Theoretical efficiency gains between 2x and 4x	Supported the hypothesis that SNN advantages may emerge on dedicated neuromorphic hardware.

Chapter 5

DISCUSSION

5.1 Threats to Validity

Several factors should be considered when interpreting the results presented in this thesis.

- Although the datasets used are widely used benchmarks within both deep learning and neuromorphic computing, they do not cover the full range of applications for which SNNs may be relevant. Consequently, the conclusions obtained in this work should be interpreted primarily in the context of visual classification tasks rather than as general statements about all possible ANN and SNN applications.
- The SNN architectures evaluated in this thesis were intentionally kept relatively simple in order to isolate the effects of encoding strategies, temporal parameters and spike dynamics. Therefore, part of the observed performance gap between ANNs and SNNs may be attributable to architectural constraints rather than to intrinsic limitations of spike-based computation.
- As a result, the measured execution time and energy consumption do not reflect the behavior of dedicated neuromorphic processors. Although theoretical MAC/SOP analyses were performed to estimate potential efficiency gains, these estimates remain approximations

5.2 Performance Differences Between Static and Temporal Datasets

One of the clearest findings of the project is that SNN performance depends strongly on the nature of the input data. On simple static datasets such as MNIST, SNNs were capable of achieving results close to equivalent ANN baselines. However, the fact that SNNs performed well on MNIST mainly validated the correctness of the implementation rather than demonstrating a strong advantage of spike-based computation.

The limitations of SNNs became more visible when moving to CIFAR-10. In this case, the ANN baseline clearly outperformed the SNN. Static images do not contain intrinsic temporal structure, so SNNs must artificially expand them across timesteps through rate encoding or similar mechanisms. This process introduces redundancy, stochastic variation and possible information loss.

It would then sound obvious that on temporal datasets SNNs outperformed ANNs, but why did N-MNIST favour SNNs but CIFAR10-DVS not? In N-MNIST, the shapes are highly structured, centered and visually separable. This makes the temporal information easier to integrate through LIF neuron dynamics. CIFAR10-DVS, in contrast, is much more complex. The event representation is not automatically easy to exploit. To fully exploit complex neuromorphic datasets, more advanced SNN architectures are likely required, such as residual spiking networks, recurrent temporal modules, attention mechanisms or event-native processing approaches.

Therefore, the results suggest that event-based data provides a necessary but not sufficient condition for SNN advantages.

5.3 Relationship Between Sparsity and Learning

Results also show that sparsity alone is not sufficient to guarantee good performance. Extremely sparse models may fail to transmit enough information, while overly active models may generate redundant spikes that increase computation without improving accuracy. The spike probability scaling experiments on MNIST showed this trade-off clearly as the best results were not obtained by maximizing spike activity, but by finding an intermediate region where the network generated enough spikes to learn effectively.

Spike activity is not determined only by the input encoding strategy. It also emerges from the interaction between input spike density, learned synaptic weights, membrane dynamics and firing thresholds. For example, increasing input spike probability did not necessarily increase output firing rate. This demonstrates that spike dynamics are adaptive and cannot be interpreted only from the input representation.

5.4 Optimization Challenges in SNNs

The loss curves obtained in the CIFAR10-DVS experiments reveal one of the main practical limitations of SNNs: optimization remains more difficult than in conventional ANNs.

This behavior is consistent with the known difficulties of training SNNs. The spike function is non-differentiable, so training relies on surrogate gradients. Although surrogate gradients make back-propagation possible, they only approximate the true derivative of the spike-generation mechanism. As a result, gradient flow is less direct than in conventional ANNs, where activations are continuous and differentiable.

Furthermore, SNNs introduce additional sources of complexity. The network must learn not only spatial feature representations, but also how membrane potentials evolve over time, when neurons should fire and how spike activity should be distributed across the simulation window. This creates a harder optimization problem than standard CNN training.

The introduction of tDBN-inspired normalization significantly improved training stability and accuracy. Normalization helped regulate internal activity, allowing neurons to operate closer to useful firing regimes and improving surrogate-gradient learning.

The results therefore support a balanced interpretation: ANNs currently remain stronger general-purpose models for classification accuracy, while SNNs offer a different computational paradigm whose advantages become most relevant under temporal, sparse and hardware-aware conditions.

5.5 Accuracy-Efficiency Trade-Off

The experiments show a persistent trade-off between accuracy and computational cost. Several modifications improved SNN performance, including normalization and deeper architectures. However, these improvements usually increased training time and computational demand.

This is especially relevant because computational efficiency is one of the main motivations for studying SNNs. If achieving competitive accuracy requires substantially more timesteps, deeper architectures or longer training, part of the theoretical efficiency advantage may be reduced.

The static dataset experiments illustrate this clearly. Increasing timesteps increased computational cost almost linearly, but did not consistently improve accuracy. Since static images contain no intrinsic temporal information, longer simulations mainly introduced redundant computation.

For temporal datasets, the trade-off was more favorable but still present. N-MNIST achieved strong performance with sparse activity. CIFAR10-DVS, however, required more complex processing and still

underperformed compared with the ANN baseline. Therefore, the advantage of SNNs depends on achieving an appropriate balance between temporal representation, architecture complexity and spike sparsity.

5.6 Hardware Dependency of SNN Efficiency

SNNs are theoretically efficient because they operate through sparse event-driven computation. However, conventional CPUs and GPUs are optimized for dense tensor operations. When SNNs are simulated in PyTorch or snnTorch, the hardware still executes operations across timesteps even if many neurons do not spike. Therefore, sparse activity does not automatically translate into lower measured energy consumption.

This explains the apparent contradiction between the CodeCarbon measurements and the operation-count analysis. CodeCarbon measures actual execution on conventional hardware, where temporal simulation overhead dominates. The operation-count analysis estimates theoretical arithmetic cost under neuromorphic assumptions, where inactive neurons do not trigger computation and spike-based synaptic operations are cheaper than dense MAC operations. The corrected MAC/SOP analysis estimated theoretical energy reductions for static CIFAR-10, N-MNIST and CIFAR10-DVS. This suggests that, if implemented on event-driven neuromorphic hardware, the observed sparsity could translate into genuine computational savings.

Chapter 6

CONCLUSIONS

6.1 Main Findings

The objective of this thesis was to investigate the behavior of Spiking Neural Networks and compare their performance with conventional ANNs. Beyond predictive accuracy, the work aimed to analyze spike activity, temporal processing behavior and computational efficiency in order to better understand the practical strengths and limitations of modern SNNs. The results allow the research questions introduced in the first chapter to be addressed directly.

RQ1: How do ANNs and Spiking Neural Networks compare on static and event-based visual datasets under comparable architectural conditions?

The experiments showed that ANN models generally achieved higher accuracy and faster training on conventional hardware. However, SNNs produced competitive results on simpler tasks and performed particularly well on N-MNIST, where the temporal structure of the data naturally aligned with spike-based processing. On the more complex CIFAR10-DVS dataset, the ANN baseline remained superior, indicating that temporal data alone does not guarantee an advantage for SNNs.

RQ2: How do timesteps, encoding strategies, membrane parameters and normalization techniques influence SNN behavior?

The results demonstrated that encoding plays a critical role in overall performance. Alternative encoding strategies produced only limited improvements over standard rate encoding, suggesting that information representation remains a major challenge for static-data SNNs. Increasing the number of timesteps did not necessarily improve accuracy and often introduced redundant computation. Among the optimization techniques investigated, normalization and tdbn provided the most consistent benefits by improving training stability and regulating spike activity.

RQ3: What relationship exists between sparsity, firing rate and predictive performance?

The spike analysis revealed that successful SNN operation emerges from a balance between information transmission and sparse activity. The best-performing models were neither the most active nor the most sparse. Instead, moderate firing rates combined with high but controlled sparsity consistently produced the strongest results, indicating that efficient spike-based computation depends on maintaining meaningful neural activity rather than maximizing sparsity alone.

RQ4: To what extent do energy measurements on conventional hardware align with theoretical neuromorphic efficiency estimates?

The experimental measurements obtained through CodeCarbon favored ANN models in terms of execution time and energy consumption on conventional hardware. However, the theoretical MAC/SOP analysis predicted potential energy reductions between approximately two and four times for SNNs under neuromorphic hardware assumptions. This discrepancy highlights the strong dependency of SNN efficiency on the execution platform and suggests that many of their theoretical advantages cannot yet be fully exploited on traditional von Neumann architectures.

Overall, the findings indicate that Spiking Neural Networks currently represent a promising complementary paradigm rather than a direct replacement for conventional ANNs. Their greatest potential appears in scenarios where temporal information, sparse activity and hardware-aware computation can be exploited simultaneously.

6.2 Future Work

One natural continuation of this work would involve the exploration of more advanced SNN architectures. The models implemented in this thesis were intentionally kept relatively simple in order to isolate the effects of encoding strategies, temporal parameters and spike dynamics. However, recent literature has demonstrated that residual spiking networks, recurrent temporal architectures, attention mechanisms and spike-based Transformer models can substantially improve performance on complex datasets. Evaluating these architectures could help determine whether the performance gap observed on CIFAR10-DVS is primarily architectural rather than intrinsic to spike-based computation.

Another promising direction concerns neuromorphic hardware deployment. Due to hardware accessibility constraints, all experiments were executed on conventional Apple Silicon hardware. Consequently, the event-driven computational advantages of SNNs could only be estimated theoretically through operation-count analysis. Future work involving deployment on dedicated neuromorphic platforms such as Intel Loihi, SpiNNaker or BrainScaleS would allow direct evaluation of the practical energy efficiency benefits associated with sparse spike-based computation.

The development of improved encoding strategies also remains an important challenge. Perhaps the principal bottleneck observed in SNNs was not spike-based computation itself, but the representation of information before computation begins. In static datasets, the encoding stage largely determined the quality of the temporal signal available to the network, suggesting that future progress may depend as much on better neural representations as on improved architectures.

Hybrid ANN-SNN systems could also be an interesting research path. Rather than replacing ANNs entirely, future architectures may combine conventional deep learning modules with spike-based processing components, exploiting the strengths of each paradigm. Such approaches could potentially offer favorable trade-offs between predictive performance and computational efficiency.

In conclusion, Spiking Neural Networks remain a challenging but highly promising research field. While many practical obstacles persist, their unique combination of temporal processing, sparse computation and neuromorphic compatibility positions them as a candidate for the next generation of energy-efficient intelligent systems.

Bibliography

- Amira, A., Chang, B., Swaminathan, P., Sanborn, S., Risk, L., and Softky, W. (2017). A low power, fully event-based gesture recognition system. In *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*.
- Davies, M., Srinivasa, N., Lin, T. H., China, G., Cao, Y., Chimalakonda, S. H., Fair, O., Gidra, S., Lazarewich, M., Gleeson, B., Mickle, A., Schneider, A. D., Frady, E. P., Saidalkhavi, J., Vignesh, R., Belhadj, S., Huttenlocher, D., Deiss, S. R., Lu, J., and Amir, A. (2018). Loihi: A neuromorphic manycore processor with on-chip learning. *IEEE Micro*.
- Davies, M., Wild, A., Orchard, G., Sandamirskaya, Y., Constandinou, T. G., Mitin, V., and Neftci, E. (2021). Advancing neuromorphic computing with loihi: A survey of results and outlook. *Proceedings of the IEEE*.
- Dayan, P. and Abbott, L. F. (2001). *Theoretical Neuroscience: Computational and Mathematical Modeling of Neural Systems*.
- Delbruck, T., Linares-Barranco, B., Culurciello, E., and Posch, C. (2010). Activity-driven, event-based vision sensors. In *IEEE International Symposium on Circuits and Systems (ISCAS)*.
- Dosovitskiy, A., Beyer, L., Kolesnikov, A., Weissenborn, D., Zhai, X., Unterthiner, T., Dehghani, M., Minderer, M., Heigold, G., Gelly, S., Uszkoreit, J., and Houlsby, N. (2021). An image is worth 16x16 words: Transformers for image recognition at scale. In *International Conference on Learning Representations (ICLR)*.
- Eshraghian, J. K., Ward, M., Neftci, E. O., Wang, X., Lenz, G., Dwivedi, G., Bennamoun, M., Jeong, D. S., and Lu, W. D. (2023). Training spiking neural networks using lessons from deep learning. *Proceedings of the IEEE*.
- Furber, S. (2016). Large-scale neuromorphic computing systems. *Journal of Neural Engineering*.
- Gallego, G., Delbrück, T., Orchard, G., Bartolozzi, C., Taba, B., Censi, A., Leutenegger, S., Davison, A. J., Scaramuzza, D., and Benosman, R. (2020). Event-based vision: A survey. *IEEE Transactions on Pattern Analysis and Machine Intelligence*.
- Gehrig, D., Shrestha, S. B., and Scaramuzza, D. (2020). Comparing representations in event-based visual recognition. In *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*.
- Gerstner, W. and Kistler, W. M. (2002). *Spiking Neuron Models: Single Neurons, Populations, Plasticity*.
- Gerstner, W., Kistler, W. M., Naud, R., and Paninski, L. (2014). *Neuronal Dynamics: From Single Neurons to Networks and Models of Cognition*.
- Goodfellow, I., Bengio, Y., and Courville, A. (2016). *Deep Learning*.
- He, K., Zhang, X., Ren, S., and Son, J. (2016). Deep residual learning for image recognition. In *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*.

- Kaplan, J., McCandlish, S., Henighan, T., Brown, T. B., Chess, B., Child, R., Gray, S., Radford, A., Wu, J., and Amodei, D. (2020). Scaling laws for neural language models. *arXiv preprint arXiv:2001.08361*.
- Laughlin, S. B. and Sejnowski, T. J. (2003). Communication in neuronal networks. *Science*.
- Lichtsteiner, P., Posch, C., and Delbruck, T. (2008). A 128×128 120 db 15 μ s latency asynchronous temporal contrast vision sensor. *IEEE Journal of Solid-State Circuits*.
- Maass, W. (1997). Networks of spiking neurons: The third generation of neural network models. *Neural Networks*.
- McCulloch, W. S. and Pitts, W. (1943). A logical calculus of the ideas immanent in nervous activity. *The bulletin of mathematical biophysics*, 5(4):115–133.
- Mead, C. (1990). Neuromorphic electronic systems. *Proceedings of the IEEE*.
- Merolla, P. A., Arthur, J. V., Alvarez-Icaza, R., Cassidy, A. S., Sawada, J., Akopyan, F., Jackson, B. L., Imam, N., Guo, C., Nakamura, Y., Brezzo, B., Vo, I., Emser, S. K., Appuswamy, R., Taba, B., Amir, A., Flickner, M. D., Risk, W. P., Manohar, R., and Modha, D. S. (2014). A million spiking-neuron integrated circuit with a scalable communication network and interface. *Science*.
- Neftci, E. O., Mostafa, H., and Zenke, F. (2019). Surrogate gradient learning in spiking neural networks: Bringing the power of gradient-based optimization to spiking neural networks. *IEEE Signal Processing Magazine*.
- Orchard, G., Jayawant, A., Cohen, G. K., and Thakor, N. (2015). Converting static image datasets to spiking neuromorphic datasets using saccades. *Frontiers in Neuroscience*.
- Rosenblatt, F. et al. (1962). *Principles of neurodynamics: Perceptrons and the theory of brain mechanisms*, volume 55. Spartan books Washington, DC.
- Roy, K., Jaiswal, A., and Panda, P. (2019). Towards spike-based machine intelligence with neuromorphic computing. *Nature*.
- Rueckauer, B., Lungu, I. A., Hu, Y., Pfeiffer, M., and Liu, S. C. (2017). Conversion of continuous-valued deep networks to efficient event-driven networks for image classification. *Frontiers in Neuroscience*.
- Schmidhuber, J. (2015). Deep learning in neural networks: An overview. *Neural Networks*.
- Schuman, C. D., Kulkarni, S. R., Parsa, M., Mitchell, J. P., Date, P., and Kay, B. (2022). Opportunities for neuromorphic computing algorithms and applications. *Nature Computational Science*.
- Strubell, E., Ganesh, A., and McCallum, A. (2019). Energy and policy considerations for deep learning in nlp. In *Annual Meeting of the Association for Computational Linguistics (ACL)*.
- Tavanaei, A., Ghodrati, M., Kheradpisheh, S. R., Masquelier, T., and Maida, A. (2019). Deep learning in spiking neural networks. *Neural Networks*.
- Zenke, F. and Vogels, T. P. (2021). The remarkable robustness of surrogate gradient learning for instilling complex function in spiking neural networks. *Neural Computation*.
- Zheng, H., Wu, Y., Deng, L., Hu, Y., and Li, G. (2020). Going deeper with directly-trained larger spiking neural networks. In *Proceedings of the AAAI Conference on Artificial Intelligence*.

Appendix A

DECLARATION OF ARTIFICIAL INTELLIGENCE USE

Artificial Intelligence tools, mainly ChatGPT, were used during the development of this thesis as supporting instruments for the following:

- Improving the clarity, grammar and phrasing of certain sections of the manuscript.
- Suggesting alternative structures and organizational improvements for the document.
- Assisting in code debugging, code refactoring and implementation troubleshooting during the experimental phase.

All scientific decisions, experimental design, implementation, analysis of results, conclusions and final written content were reviewed, validated and approved by the author. Artificial Intelligence tools were used exclusively as supportive aids and did not replace the author's intellectual contribution or critical judgement.